

Electronics for Embedded Systems notes

Luca Seggiani
2026

1 Lesson 23-09-26

1.1 Introduction

The course is meant to give an introduction to *embedded systems*, after a review of analog and digital electronics. The topics covered are:

- Memories;
- (Programmable Logic Devices) (PLDs);
- Interconnection protocols and I/O;
- Embedded processors and peripherals;
- **DA** (*Digital-to-Analog*) and **AD** (*Analog-to-Digital*) conversion;
- Power management.

1.2 Review

Let's begin the review of analog and digital electronics. The review will cover:

- Electrical components, including:
 - **Passive components**, as in *resistors*, *capacitors* and *inductors*;
 - **Active components**, as in *diodes* and *transistors*;
 - **Operational amplifiers**.
- Digital electronics, including *logic gates*, *digital functions*, **FSMs** (*Finite State Machines*) and the *VHDL* language.

1.2.1 Passive components

Resistors offer *resistance* to electric current, measured in *Ohms* (Ω). By Ohm's law:

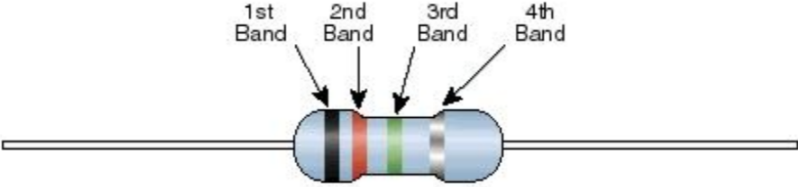
$$v_R(t) = Ri_R(t), \quad V_R(s) = RI_R(s)$$

with $s = \sigma + j\omega$ in the Laplace domain. A definition via *conductance*, measured in *Siemens* (S), is also possible:

$$i_R(t) = Gv_R(t), \quad g = \frac{1}{R}$$

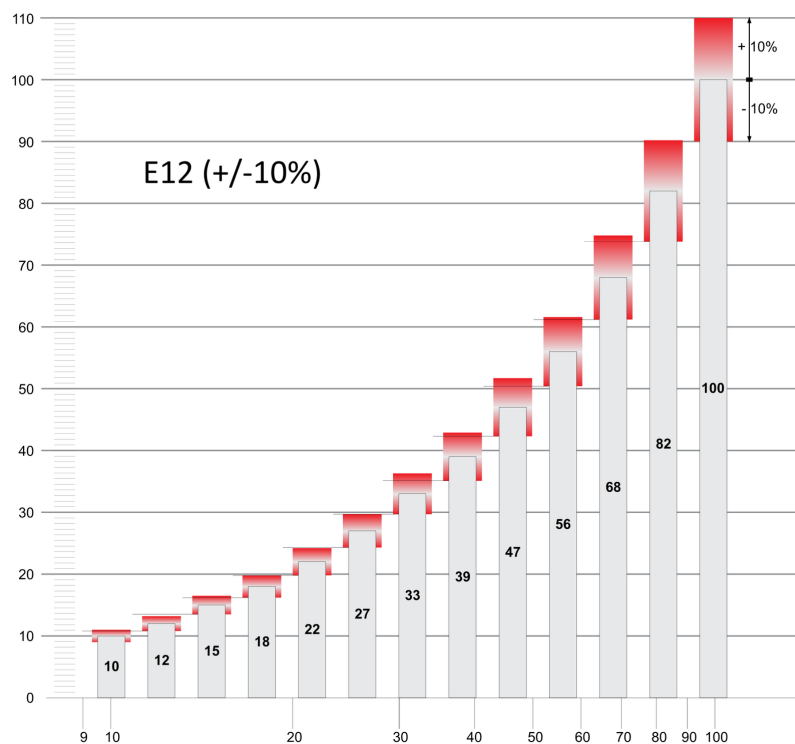
Resistor response doesn't depend on the frequency of periodic signals (all signals get attenuated) the same. This applies at least partly to real resistors, and we will assume the ideal model from now onwards.

It can also be useful to remember the code used to distinguish different resistance values.

Standard EIA Color Code Table 4 Band: $\pm 2\%$, $\pm 5\%$, and $\pm 10\%$


Color	1st Band (1st figure)	2nd Band (2nd figure)	3rd Band (multiplier)	4th Band (tolerance)
Black	0	0	10^0	
Brown	1	1	10^1	
Red	2	2	10^2	$\pm 2\%$
Orange	3	3	10^3	
Yellow	4	4	10^4	
Green	5	5	10^5	
Blue	6	6	10^6	
Violet	7	7	10^7	
Gray	8	8	10^8	
White	9	9	10^9	
Gold			10^{-1}	$\pm 5\%$
Silver			10^{-2}	$\pm 10\%$

Resistor tolerances are to be taken into account, as they might drive resistances in our circuits out of the feasible margins. Because of this, we prefer using standard values in *E series*. E series is a system of preferred numbers (also called preferred values) derived for use in electronic components. It consists of the E3, E6, E12, E24, E48, E96, and E192 series, where the number after the "E" designates the quantity of logarithmic value "steps" per decade.



Capacitors store energy as voltage potential between conductive armatures separated by a dielectric. *Capacity*, measured in *Farad* (F) is defined as:

$$C = \frac{Q}{V}$$

The laws that govern capacitors in the time domain are differential equations:

$$i_C(t) = C \frac{d}{dt} v_C(t), \quad v_C(t) = \frac{1}{C} \int_0^t i_C(\tau) d\tau$$

It is then easier to move to the Laplace domain, where we can simply express integration as division by s :

$$V_C(s) = \frac{I_C(s)}{sC}$$

In regards to periodic signals, the behaviour of capacitors depends on frequency:

- When $f \rightarrow 0$ it behaves as an *open circuit*;
- When $f \rightarrow \infty$ it behaves as a *short circuit*;

Many types of capacitors are available:

- *Electrolytic* capacitors, which are polar;
- *Ceramic* capacitors, which are not polar.

To read the value of a capacitor we have quite a complex system:

- Two digits: it is the value in pF ;
- Three digits: the first two are the value in pF , and the third is the multiplier; If the third digit is 8 the multiplier is 10^{-2} , if it is 9 the multiplier is 10^{-1} ;
- A digit, a character and a digit: the two digits are the one before and after the decimal point, the character is the unit of measure;
- A character follows the last digit indicates the tolerance, with $J = \pm 5\%$, $K = \pm 10\%$, $M = \pm 20\%$.

Inductors store energy as magnetic field in a winding of conductive wire around a metal core. *Inductance*, measured in *Henry* (H) is defined as:

$$L = \frac{\Phi}{I}$$

The laws that govern inductors in the time domain are differential equations:

$$v_L(t) = L \frac{d}{dt} i_L(t), \quad i_L(t) = \frac{1}{L} \int_0^t v_L(\tau) d\tau$$

It is then easier to move to the Laplace domain, where we can simply express derivation as multiplication by s :

$$V_L(s) = sLI_L(s)$$

In regards to periodic signals, the behaviour of inductors also depends on frequency, on the contrary of capacitors:

- When $f \rightarrow \infty$ it behaves as an *open circuit*;
- When $f \rightarrow 0$ it behaves as a *short circuit*;

1.2.2 Voltage and current dividers

We show a chart showcasing a number of elementary divider circuits which we can use.

- Voltage divider with two resistances

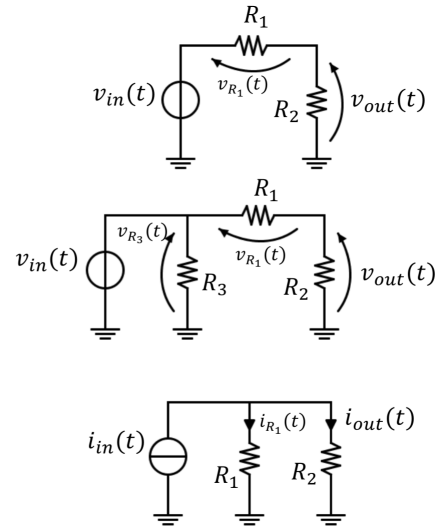
- $v_{out}(t) = \frac{R_2}{R_1 + R_2} v_{in}(t) = \frac{g_1}{g_1 + g_2} v_{in}(t)$
- $v_{R_1}(t) = \frac{R_1}{R_1 + R_2} v_{in}(t) = \frac{g_2}{g_1 + g_2} v_{in}(t)$

- Voltage divider with three resistances

- $v_{out}(t) = \frac{R_2}{R_1 + R_2} v_{in}(t) = \frac{g_1}{g_1 + g_2} v_{in}(t)$
- $v_{R_3}(t) = v_{in}(t)$

- Current divider

- $i_{out}(t) = \frac{R_1}{R_1 + R_2} i_{in}(t) = \frac{g_2}{g_1 + g_2} i_{in}(t)$
- $i_{R_1}(t) = \frac{R_2}{R_1 + R_2} i_{in}(t) = \frac{g_1}{g_1 + g_2} i_{in}(t)$



Let us note in the second example (voltage divider with three resistances) that nothing changes by adding the third resistor. $V_{out}(t)$ is exactly the same, as the potential seen by R_1 on its left side doesn't change (and the potential across R_3 is the same as $v_{in}(t)$).

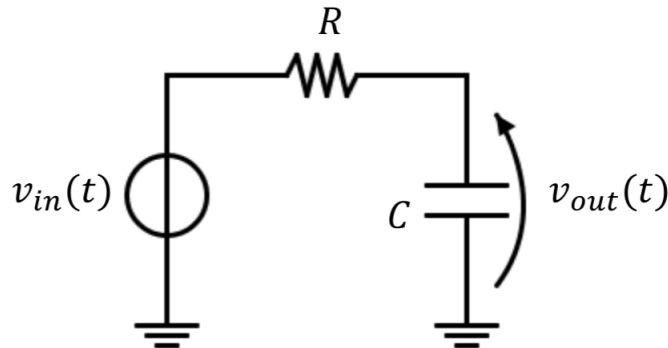
What actually does change in the example is the resistance: we have effectively built a current divider, where the input current is:

$$i_{in}(t) = \frac{v_{in}(t)}{R_1 + R_2} + \frac{v_{in}(t)}{R_3}$$

Lastly, let's note that all the equations in the Laplace domain are the same, changing the variables accordingly.

1.2.3 Simple low pass filters

A simple low pass filter can be seen as a special case of a divider:



By writing the *Kirchoff Voltage Law* (KVL) of the circuit, we get:

$$v_{in}(t) - R \cdot C \frac{d}{dt} v_{out}(t) - v_{out}(t) = 0$$

with the first term being the voltage gain on the power supply, the second term being the voltage drop on the resistor, and the last term being the output voltage taken across the capacitor.

Time domain solution This is a first order linear differential equation, which we can express as:

$$\frac{d}{dt}v_{out}(t) + \frac{1}{\tau}v_{out}(t) = \frac{1}{\tau}v_{in}(t), \quad \tau = RC$$

We refer to $\tau = RC$ as the *time constant* of the circuit. A general solution is:

$$v_{out}(t) = e^{-\frac{t}{\tau}} \left(k + \int \left(\frac{v_{in}(t)}{\tau} e^{\frac{t}{\tau}} \right) dt \right)$$

It can be useful to carry out the full calculation. We start from the general differential equation obtained from KVL:

$$v_{in}(t) - RC \frac{d}{dt}v_{out}(t) - v_{out}(t) = 0$$

We can easily rearrange this into a standard first-order form:

$$\frac{d}{dt}v_{out}(t) + \frac{1}{RC}v_{out}(t) = \frac{1}{RC}v_{in}(t)$$

Now we take a function $\mu(t)$ constructed such that:

$$\mu(t) \frac{1}{RC} = \frac{d}{dt}\mu(t)$$

While the purpose of this might at this point be unclear, we can clearly see that a function that satisfies the equation is:

$$\mu(t) = e^{\frac{t}{RC}}$$

Let's multiply both sides of the original differential equation by $\mu(t)$:

$$\mu(t) \frac{d}{dt}v_{out}(t) + \mu(t) \frac{1}{RC}v_{out}(t) = \mu(t) \frac{1}{RC}v_{in}(t)$$

By applying the property above this leaves us with:

$$\mu(t) \frac{d}{dt}v_{out}(t) + \frac{d}{dt}\mu(t)v_{out}(t) = \frac{\mu(t)}{RC}v_{in}(t)$$

It's clear that the left hand side is the expansion of the derivative of a product, so we can write:

$$\frac{d}{dt}(\mu(t)v_{out}(t)) = \frac{\mu(t)}{RC}v_{in}(t)$$

Recall that we were interested in $v_{out}(t)$. We can take it out the derivative by integrating both sides:

$$\mu(t)v_{out}(t) = \int \frac{\mu(t)}{RC}v_{in}(t) + k$$

where the antiderivative operation requires that we include a $\pm k$ factor (sign indifferent).

At this point we just have to make $v_{out}(t)$ explicit and substitute our original definition of $\mu(t)$:

$$v_{out}(t) = \frac{1}{\mu(t)} \left(\int \frac{\mu(t)}{RC} v_{in}(t) + k \right) = e^{-\frac{t}{RC}} \left(k + \int \left(\frac{v_{in}(t)}{RC} e^{\frac{t}{RC}} \right) dt \right)$$

Which is our original expression (given $\tau = RC$). $\square$

With $v_{in}(t)$ taken as a step of amplitude V_M , or equivalently by taking the solution given with zero initial conditions and a constant power source, we get the usual form:

$$v_{out}(t) = V_M(1 - e^{-\frac{t}{\tau}})$$

where the time constant determines the time taken to reach saturation. We recall that in one time constant we reach around 63.2% of full saturation, and in three time constants we reach 95%.

Laplace domain solution Let's carry out the same calculation in the Laplace domain. By expressing the voltage divider we get:

$$V_{out}(S) = \frac{\frac{1}{sC}}{\frac{1}{sC} + R} V_{in}(S) = \frac{1}{sRC + 1} V_{in}(s)$$

where in the first expression we simply applied the voltage divider law to the impedance definitions of the capacitor and the resistor. We can then derive a *transfer function*:

$$\frac{V_{out}(s)}{V_{in}(s)} = \frac{1}{sRC + 1}$$

We can shift to the Fourier domain, or in equivalent terms take a projection of the Laplace domain in only the frequency direction, by setting $s = j\omega$:

$$\frac{V_{out}(j\omega)}{V_{in}(j\omega)} = \frac{1}{j\omega RC + 1}, \quad \left| \frac{V_{out}(j\omega)}{V_{in}(j\omega)} \right| = \frac{1}{\sqrt{\omega^2 R^2 C^2 + 1}}$$

A *pole* in this expression is a zero of the denominator, so:

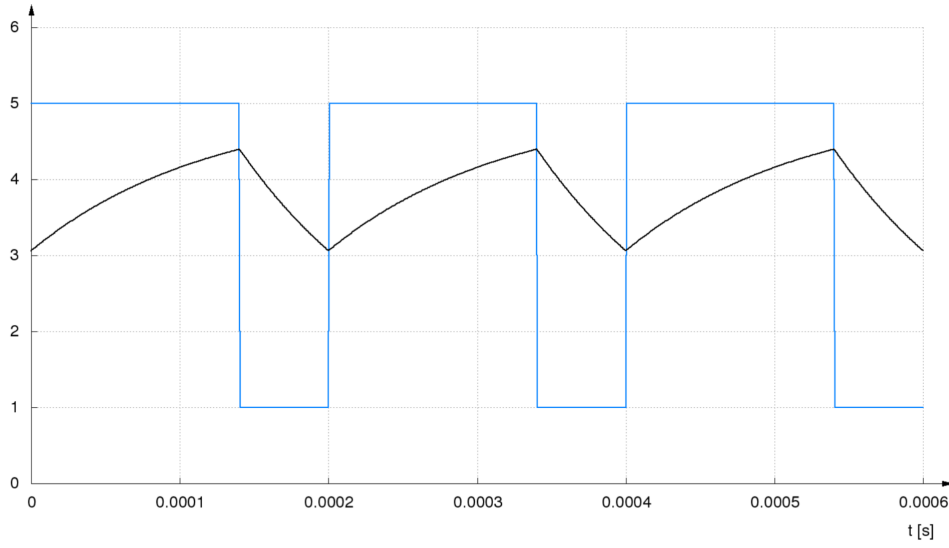
$$s_p = -\frac{1}{RC}, \quad \omega_p = \frac{1}{RC}$$

with the frequency representation being bilateral, as always. Let's note that the *pole pulsation* ω_p is not really a zero of the denominator, but just the characteristic frequency of the system, which we find useful for plotting and solving circuit responses.

Time domain response Fixing V_{in} to a periodic square wave, let's compute the time domain response. Each fragment in the response is an exponential:

$$v_{out}(t) = A + B e^{-\frac{t}{\tau}}$$

where $\tau = RC$.



We can proceed with the following approach:

1. Evaluate the waveforms at the extreme, fixing initial conditions;
2. Determine A and B for the rising and falling cases;
3. Determine the maximum and minimum voltages.

And the result we obtain should be valid at the steady-state. We say *steady-state*, as when the oscillators first begins oscillating there will be a transient, which will temporarily not obey to our prediction.

Let's start with the rising edge. By remembering our first equation, we can compute V_{out} at the first time instant:

$$V_{out}^r(0) = A + B = V_{min}$$

If at this point the square wave never went to a low value, the capacitor would continue charging and we would approach saturation (at V_H , the maximum value of the square wave). From this we can derive $A = V_H$, as the B term vanishes to 0 approaching infinity, and it follows that $B = V_{min} - V_H$. This gives us:

$$V_{out}^r(t) = V_H + (V_{min} - V_H)e^{-\frac{t}{\tau}}$$

What happens in reality is that at some time T_H we switch to the falling edge, reaching a maximum value V_{max} .

$$V_{out}^r(T_H) = A + Be^{-\frac{T_H}{\tau}} = V_H + (V_{min} - V_H)e^{-\frac{T_H}{\tau}} = V_{max}$$

To fix V_{min} and V_{max} , we switch to analyzing the falling edge. With similar calculations, we can derive the equation:

$$V_{out}^f(T_L) = V_L + (V_{max} - V_L)e^{-\frac{T_L}{\tau}} = V_{min}$$

with V_L at the lowest point of the square wave.

We can easily solve the resulting system of equations, to figure out the minimum and maximum bounds of the response of the circuit:

$$\begin{cases} V_H + (V_{min} - V_H)e^{-\frac{T_H}{\tau}} = V_{max} \\ V_L + (V_{max} - V_L)e^{-\frac{T_L}{\tau}} = V_{min} \end{cases}$$

By setting:

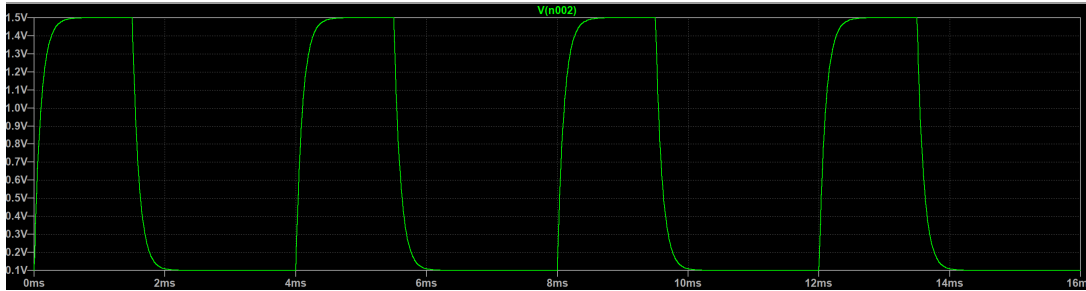
$$\alpha = e^{-\frac{T_H}{T}}, \quad \beta = e^{-\frac{T_L}{T}}$$

we get the solutions (calculations omitted because they're boring):

$$V_{max} = \frac{V_H (1 - \alpha) + \alpha V_L (1 - \beta)}{1 - \alpha\beta}, \quad V_{min} = \frac{\beta V_H (1 - \alpha) + V_L (1 - \beta)}{1 - \alpha\beta}$$

We can then trace the actual response by drawing exponential branches with the time constant given by $\tau = RC$.

Lastly, let's see an LTSPICE (simulation program) plot of the square wave response of this circuit, which confirms our mathematical model:



Frequency domain response In the frequency domain, we can consider the effect on a periodic signal such as a sine wave. Let's have:

$$V_{in}(t) = M \sin(\omega t) = M \sin(2\pi f t)$$

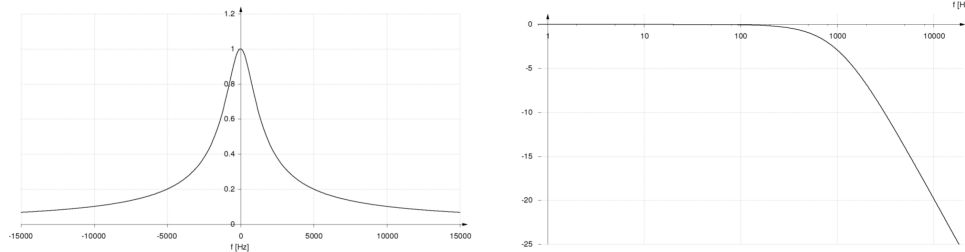
In a linear system at the steady state (which we have) if the input is sinusoidal, the output is sinusoidal at the same frequency: only the amplitude and the phase can be shifted:

$$V_{out}(t) = M' \sin(\omega t + \phi)$$

By using the Fourier domain equations calculated earlier, we can actually express M' and ϕ :

$$M' = M \frac{\omega_p}{\sqrt{\omega^2 + \omega_p^2}}, \quad \phi = -\tan^{-1} \left(\frac{\omega}{\omega_p} \right)$$

For arbitrary frequencies, we can plot the linear and logarithmic Bode diagrams of amplitude:



Let us note that at $f_p = \frac{\omega_p}{2\pi}$ we have the -3 dB band for our transfer function, which we can also refer to as the *cutoff frequency* or *pole frequency* of the lowpass.

1.2.4 Superposition of effects

Theorem 1.1: Superposition of effects

In a linear system with more than one input (independent ideal sources, voltage or current), the output is the algebraic sum of the effects of each input with the other set to 0.

This means:

- Voltage source at zero $\rightarrow$ short circuit;
- Current source at zero $\rightarrow$ open circuit;
- Dependent sources must not be set to 0.

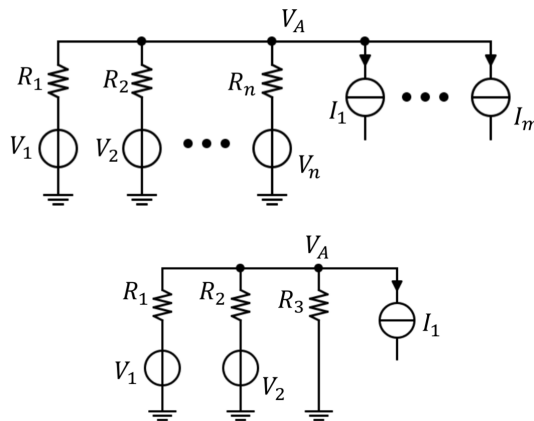
Superposition applies to *linear* systems, that is circuits which feature:

- Sources (independent and dependent), resistors, capacitors, inductors, transformers;
- Non linear system

If diodes, transistors, operational amplifiers or other active elements are present, the superposition of effects is not applicable. In these cases we need to first *linearize* the system around a bias point.

1.2.5 Millman's theorem

Let us take the circuit:



Theorem 1.2: Millman's Theorem

In the general network on the right, the voltage V_A is given by the expression:

$$V_A = \frac{\sum_{i=1}^n \frac{V_i}{R_i} - \sum_{j=1}^m I_j}{\sum_{i=1}^n \frac{1}{R_i}}$$

The theorem can be proved using the superposition of effects and voltage dividers with conductances. Let us work through it.

The idea is to:

1. Calculate the contribution on voltage V_A given by just a voltage generator V_j ; then we can sum the contribution over all voltage generators to get the cumulative voltage generator contribution, which we'll call V_A^a ;

2. Calculate the contribution on voltage V_A given by just a current generator I_j ; then we can sum the contribution over all current generators to get the cumulative current generator contribution, which we'll call V_A^b .

Superposition is applied in both steps, and one final time to sum the contributions of the voltage and current generators and get the final expression for V_A .

- For the voltage generators, we take the contribution of a single voltage generator V_j as:

$$V_A^{a_j} = \frac{\parallel_{i \neq j}^n R_i}{\parallel_{i \neq j}^n R_i + R_j} V_j$$

where $\parallel_{i \neq j}^n R_i$ is notation soup for the parallel of all resistance but R_j .

We can sum over these to get the total voltage generator contribution V_A^a :

$$V_A^a = \sum_{j=1}^n \frac{\parallel_{i \neq j}^n R_i}{\parallel_{i \neq j}^n R_i + R_j} V_j$$

This sum is a bit hard to compute, as the denominator changes at each value of j and we don't have a clean way to expand it. Things would be quite a bit easier if we had the same denominator at each value of j . We can make this happen by dividing the numerator and the denominator by $R_j \parallel_{i \neq j}^n R_i$, which gives:

$$V_A^a = \sum_{j=1}^n \frac{\frac{1}{R_j}}{\frac{1}{R_j} + \frac{1}{\parallel_{i \neq j}^n R_i}} V_j$$

To our convenience, we note that $\frac{1}{\parallel_{i \neq j}^n R_i}$ is equal to $\sum_{i \neq j}^n \frac{1}{R_i}$, so:

$$V_A^a = \sum_{j=1}^n \frac{\frac{1}{R_j}}{\frac{1}{R_j} + \sum_{i \neq j}^n \frac{1}{R_i}} V_j = \sum_{j=1}^n \frac{\frac{1}{R_j}}{\sum_{i=1}^n \frac{1}{R_i}} V_j = \frac{\sum_{i=1}^n \frac{V_i}{R_i}}{\sum_{i=1}^n \frac{1}{R_i}}$$

- Current generators are thankfully simpler to handle. We take the contribution of a single current generator I_j as:

$$V_A^{b_j} = I_j \parallel_{i=1}^n$$

We can sum over these to get the total voltage generator contribution V_A^b :

$$V_A^b = \sum_{j=1}^m I_j \parallel_{i=1}^n = \sum_{j=1}^m \frac{I_j}{\sum_{i=1}^n \frac{1}{R_i}} = \frac{\sum_{j=1}^m I_j}{\sum_{i=1}^n \frac{1}{R_i}}$$

At this point we just sum to get the final result. □

1.2.6 Active components

Diodes are non-linear active components, built today using solid state technologies, and more specifically *PN junctions* (as opposite to tubes, which were historically used).

By combining two doped regions of semiconductors (usually Silicon), respectively P (boron) and N (arsenic or phosphorous) doped, we obtain an initial condition where:

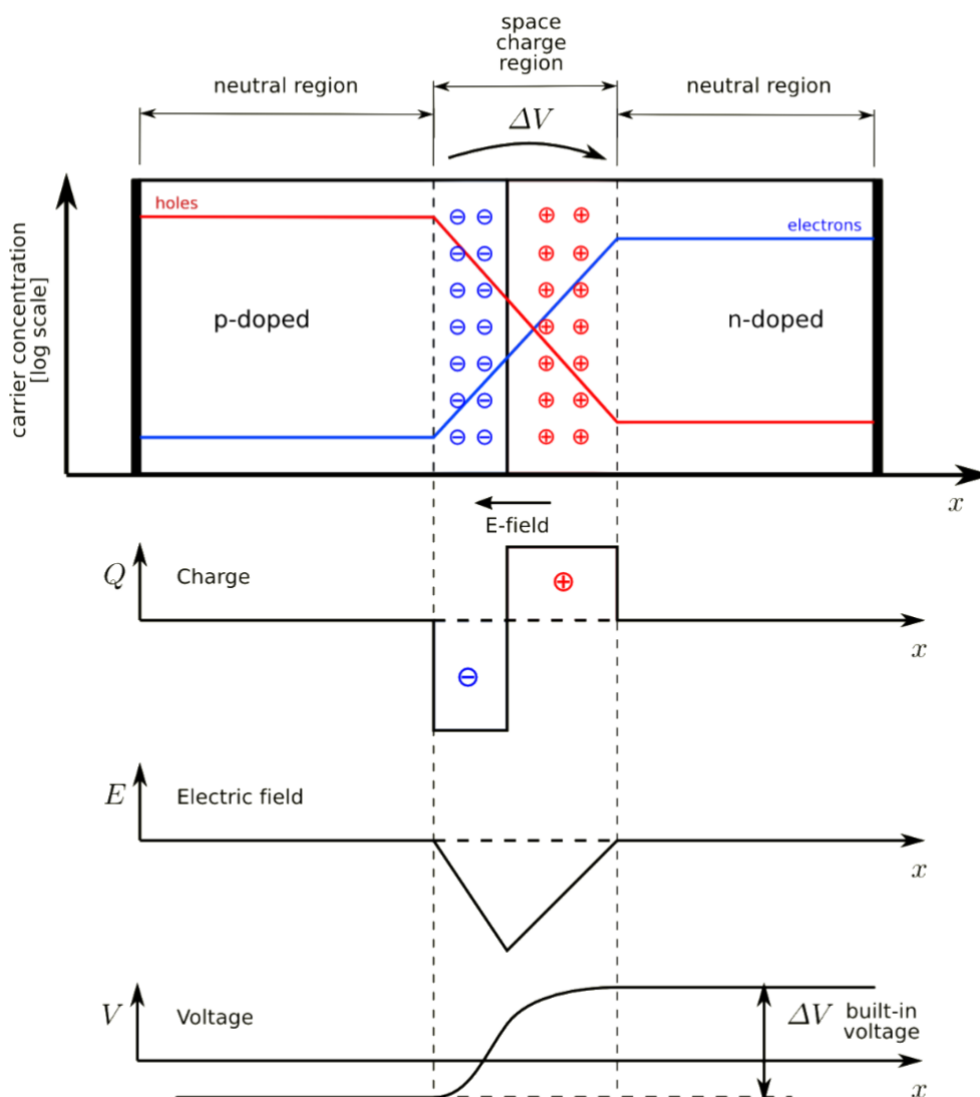
- Electrons from the n side flow by diffusion to the p side: fixed positive ions are left in the n side.

- Holes from the p side flow by diffusion to the n side: fixed negative ions are left in the p side.

At thermal equilibrium:

- Fixed ions in the two sides create an electrical field which opposes the further diffusion of electrons and holes, this is called a *depletion layer* or *space charge region*;
- A built-in voltage V_{bi} is established;
- The built-in voltage opposes current, acting as a barrier, or *threshold*, which must be overcome in order to have current flowing.

The usual value of the voltage needed to get current flowing through a diode is around $V_\gamma \approx 0.7V$.



Applying a voltage to the junction, that is applying a *forward bias*, we get multiple effects around V_γ :

- No current flows if $v_D < V_\gamma$;
- Current starts flowing if $v_D > V_\gamma$.

In *reverse bias*, no current flows and depletion layer width increases. For large reverse voltages, we see *breakdown* of the junction, followed by a large reverse current.

This effect is the combined result of *tunneling effect* or *avalanche effect*.

There is no damages to the diode, but the large current may increase power dissipation and heat, causing damages. This effect is exploited in Zener diodes, which break at specific reference voltages.

2 Lesson 24-09-26

We continue treating *active components*, starting from the **current-voltage** characteristic of the Diode:

Definition 2.1: Schottky's law

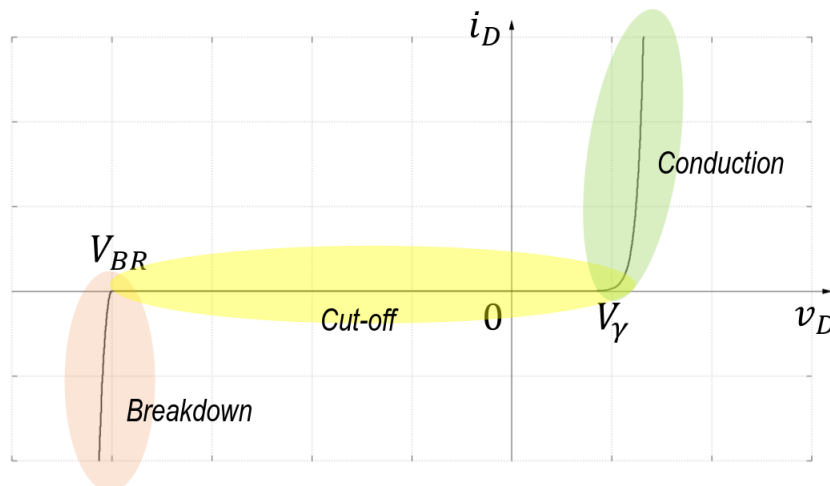
The current-voltage characteristic of a diode is:

$$i_D(t) = I_S \left(e^{\frac{v_D(t)}{\eta V_T}} - 1 \right)$$

Where:

- I_S is the *saturation current*, interpreted as the maximum reverse current we can force through the diode;
- η is the *ideality factor*, between 1 and 3 (usually simplified to 1);
- V_T is the *voltage equivalent of temperature*, around 26 millivolts.

Of course, the breakdown effect experienced by the diode for $v_D(t) \ll 0$ isn't taken into account in Schottky's law. We usually take approximated models where the diode starts acting like a voltage source if the reverse current conduction constraint is satisfied.



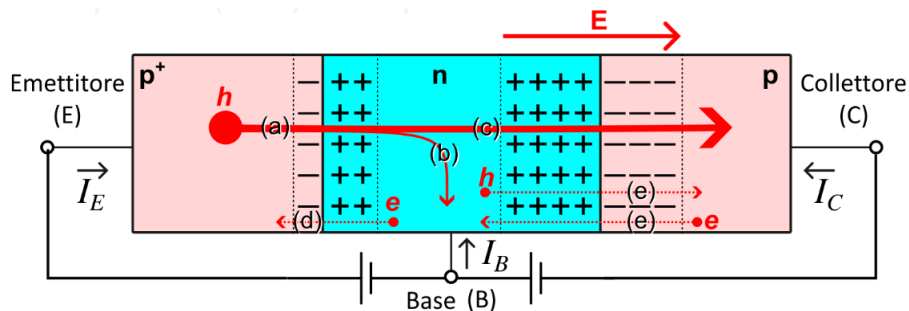
Diodes can be *modeled* in a variety of simplified ways:

- **Ideal switch:** when voltage drop is negative, current is 0; when current is positive, the voltage drop is 0;
- **Ideal switch with offset:** when voltage drop is less than V_γ , the current is 0; when current is positive, the voltage drop is V_γ ;

- **Ideal switch** with *dynamic resistance*: when voltage drop is less than V_γ , the current is 0; when voltage drop is more than V_γ , the current follows a resistive law. This is equivalent to applying a small series resistance when the ideal diode is conducting.

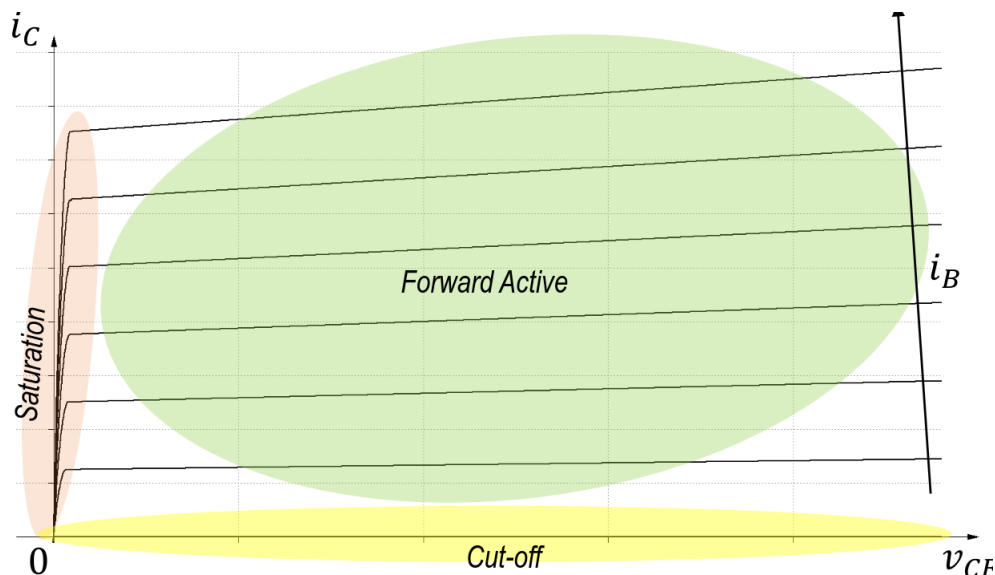
The way circuits are solved by using approximate models is by first making an *assumption* about the state of a diode (conducting or not), and then checking that assumption. Since the models have 2 states, for a diode we can assume that if it's not one state, it's the other. For n diodes we have to make n choices, and thus explore a 2^n assumption space.

BJT transistors are active semiconductor components, commonly used as amplifiers or current controlled switches. They also are a non-linear component, built from 2 PN junctions put head-to-head (which we know on their own made diodes). Two types of BJT transistors exist: PNP and NPN. NPNs are more common.



The BJT transistor is *tripole* made up of:

- **Emitter**: emits charge carriers scaled by the base current;
- **Base**, used to control the device via current;
- **Collector**, collects charge carriers emitted by the emitter.



BJTs work in 3 main regions:

- **Forward active** region, where the collector current is βi_B , so a direct scaling of the base current by a gain factor β . The voltage drop of the base on the emitter is taken with the usual diode law as $V_\gamma \approx 0.7V$. A *reverse active* region also exists but is usually undesirable;

- **Cut-off** region, where the collector current is 0 and the BJT acts as an open switch;
- **Saturation** region, where the v_{CE} between collector and emitter is approximately 0 (around $0.1 \sim 0.2V$). In this region the collector cannot give more current and the BJT acts as a closed switch.

MOS transistors are also active semiconductor components, like BJTs, commonly used as amplifiers or current controlled switches. They use a different technology to obtain basically the same characteristics: historically they were proposed around the time BJTs started becoming popular, but weren't implemented until much later.

The main difference is that MOS transistors act as *voltage controlled*, instead of *current controlled* devices.

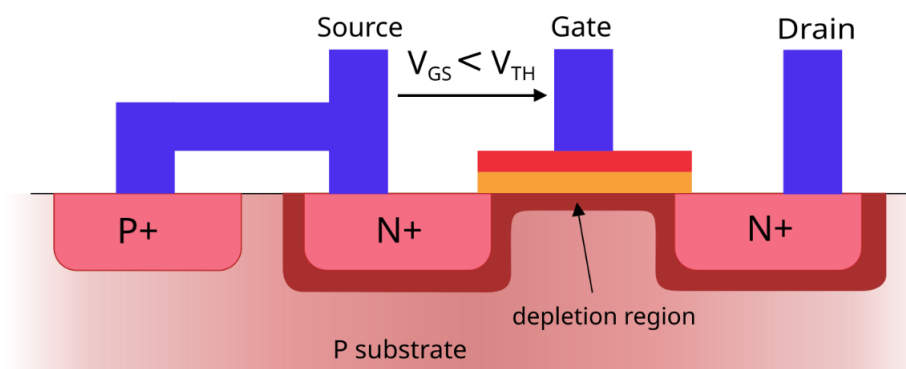
MOSes consist, briefly, of three superimposed layers:

- A base layer of doped semiconductor, which is usually silicon (Si) doped P;
- An intermediate oxide layer, which historically was silicon oxide (SiO_2) and today is mainly polysilicate. In general, we assume that the oxide is a perfect insulator (in reality, it never is, but this approximation is useful to us);
- An upper metal layer, which historically was aluminum (Al).

There are then two additional electrodes, connected to N-doped regions in the P-type substrate. We call these electrodes **source** and **drain**. At this point, the bottom electrode G is called **gate**.

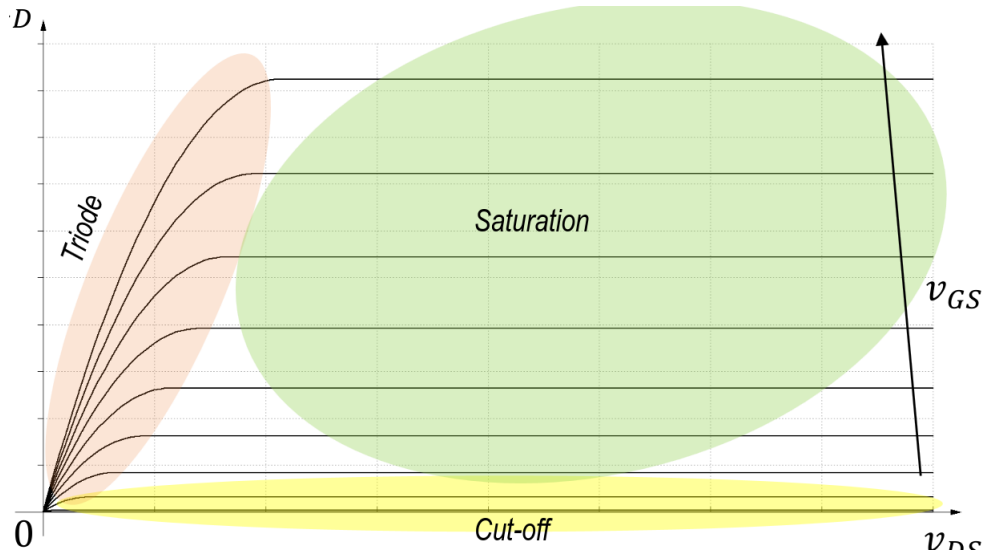
We can see that between the P-type substrate and the two N-type electrodes (source and drain), *diodes* are essentially formed, preventing current from flowing from the source and drain into the substrate.

For the MOSFET to operate, therefore, the N-doped regions need to extend beneath (or at least slightly overlap) the oxide. We call the distance L between the two N-doped regions the *gate length*.



So, we can say that a MOS transistor is another *tripole* made up of:

- **Source**: emits charge carriers scaled by the base voltage;
- **Gate**, used to control the device via voltage, it is always isolated from drain and source;
- **Drain**, collects charge carriers emitted by the emitter.



MOSes work in 3 main regions:

- **Cut-off** region: when the v_{GS} voltage between gate and source is lower than a certain v_{th} threshold voltage, the drain current is zero and the MOS behaves as an open circuit;
- **Triode** or *linear* region: when the v_{GS} voltage is higher than v_{th} , but v_{DS} is less than $v_{GS} - v_{th}$, that is:

$$v_{GS} > v_{th}, \quad v_{DS} < v_{GS} - v_{th}$$

then we are in a linear region where current scales mostly directly with voltage applied to v_{DS} . The exact law is:

$$i_D = \mu C_{ox} \frac{W}{L} \left((v_{GS} - v_{th})v_{DS} - \frac{v_{DS}^2}{2} \right)$$

where C_{ox} is the oxidation coefficient of the oxide (usually a polysilicate), and W and L are respectively the width and the length of the MOS gate channel;

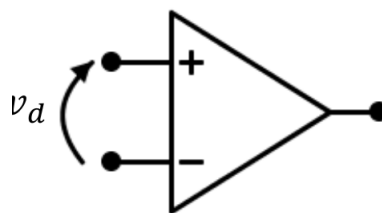
- **Saturation** region: when the v_{GS} voltage is higher than v_{th} , but v_{DS} is higher than $v_{GS} - v_{th}$, that is:

$$v_{GS} > v_{th}, \quad v_{DS} > v_{GS} - v_{th}$$

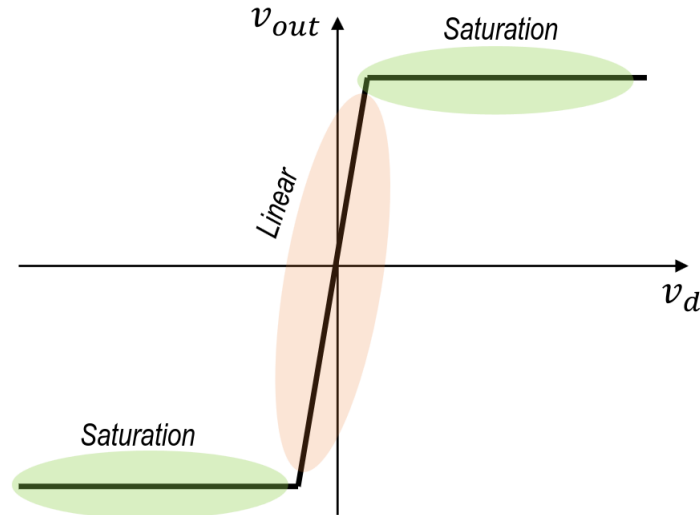
then we are in saturation, where the current scales directly with v_{GS} independently of v_{DS} . The exact law is:

$$i_D = \frac{1}{2} \mu C_{ox} \frac{W}{L} (v_{GS} - v_{th})^2$$

Operational amplifiers are complex components comprised of DC-coupled differential amplifiers with very high gain values.



What OP amps basically do is take the difference in voltage between two terminals (usually labeled + and -) and amplify that difference with an extremely high open loop gain (G_{OL}).



This means the output characteristic is:

$$v_{out} = G_{OL}(v^+ - v^-) = G_{OL}v_D$$

Let us note that, even if G_{OL} is in the order of several millions, an OP amp is a powered device which requires power and ground rails, and that cannot return voltages lower than the ground rail or higher than the power rail (a phenomenon known as *saturation*).

They are most commonly used as a **linear amplifier**:

- With a negative feedback network;
- Forced to work in linear region;
- Lower final gain, larger bandwidth, stability.

Or used as a **comparator**:

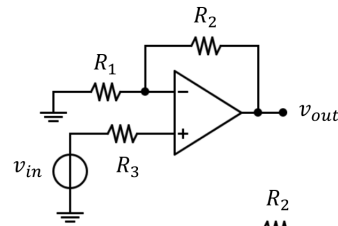
- With a positive feedback network (or no feedback);
- Forced to work in saturation regions.

They are widely used in many embedded systems.

Below, a few common circuits with OP amps are noted.

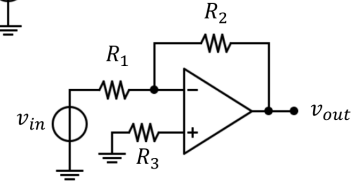
- Non inverting amplifier

- $v_{out} = v_{in} \left(1 + \frac{R_2}{R_1}\right)$, $R_{in} = \infty$, $R_{out} = 0$
- R_3 used to reduce unwanted output offsets



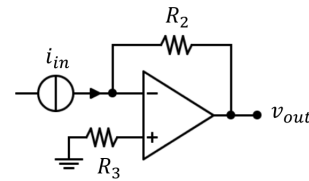
- Inverting amplifier

- $v_{out} = v_{in} \left(-\frac{R_2}{R_1}\right)$, $R_{in} = R_1$, $R_{out} = 0$
- Note the non ideal (low) input resistance



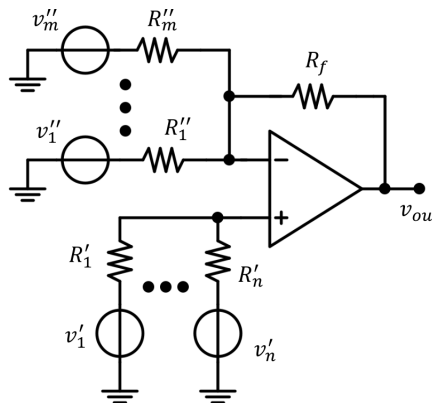
- Transresistance amplifier

- $v_{out} = -R_2 i_{in}$, $R_{in} = 0$, $R_{out} = 0$



We note that transresistance amplifiers will be useful to us when we need to implement analog converters.

Generalized analog adders / subtractors can be designed via OP amps by connecting inputs we want to add to the non inverting input and inputs we want to subtract to the inverting input.



Let's make sure to clearly define what we're after before delving into the implementation details. We ideally want a network based on a negative feedback OP amp amplifier, with a single R_f resistance as the feedback network.

- On the non-inverting input, we'll connect in parallel n voltage generators $v'_1, v'_2, \dots, v'_n$, with series resistances of $R'_1, R'_2, \dots, R'_n$;
- On the inverting input, we'll connect in parallel m voltage generators $v''_1, v''_2, \dots, v''_m$, with series resistances of $R''_1, R''_2, \dots, R''_m$.

We want the output of this network to perform the following operation on the voltage sources, taken as inputs:

$$V_{out} = \sum_{i=1}^n A'_i v'_i - \sum_{i=1}^m A''_i v''_i$$

- Let's recall that the inverting input gain can be taken as:

$$V_{out_j}'' = -\frac{R_f}{R_j''} v_j''$$

- Whereas the non-inverting input gain can be taken as:

$$V_{out_j}' = \left(1 + \frac{R_f}{\prod_{i=1}^m R_i''}\right) V_j'$$

The idea is to choose the resistances R_i' and R_i'' such that the expression of V_{out} above holds.

- Choosing the resistances on the inverting side is really easy as we can simply say:

$$R_i'' = \frac{R_f}{A_i'}$$

and in the inverting input gain equation we get the correct gain;

- For the inverting side, we first compute the voltage on node V^+ through Millman's theorem (theorem 1.2, of which we gave a proof in section 1.2.5):

$$V^+ = \frac{\sum_{i=1}^n \frac{v_i'}{R_i'}}{\sum_{i=1}^n \frac{1}{R_i'}}$$

From this equation we see that a good starting point is setting:

$$R_i' = \frac{R_f}{A_i''}$$

so that the gains at the numerator are correct:

$$V^+ = \frac{\sum_{i=1}^n A_i' v_i'}{\sum_{i=1}^n A_i'}$$

Now we use the non-inverting input gain to compute the output of the non-inverting side:

$$\begin{aligned} V_{out}' &= V^+ \left(1 + \frac{R_f}{\prod_{i=1}^m R_i''}\right) = \frac{\sum_{i=1}^n A_i' v_i'}{\sum_{i=1}^n A_i'} \left(1 + \frac{R_f}{\prod_{i=1}^m \frac{1}{R_i''}}\right) \\ &= \frac{\sum_{i=1}^n A_i' v_i'}{\sum_{i=1}^n A_i'} \left(1 + R_f \sum_{i=1}^m \frac{1}{R_i''}\right) = \frac{\sum_{i=1}^n A_i' v_i'}{\sum_{i=1}^n A_i'} \left(1 + \sum_{i=1}^m A_i''\right) \end{aligned}$$

At this point we impose the output be equal to the positive side of the original output equation:

$$\frac{\sum_{i=1}^n A_i' v_i'}{\sum_{i=1}^n A_i'} \left(1 + \sum_{i=1}^m A_i''\right) = \sum_{i=1}^n A_i' v_i' \implies \frac{1 + \sum_{i=1}^m A_i''}{\sum_{i=1}^n A_i'} = 1$$

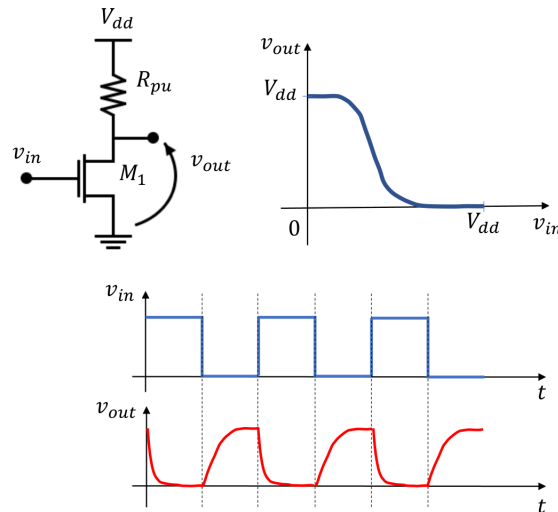
This last equation we obtained is what we have to satisfy in order to get the correct resistor values. It's pretty simple to make the result right by just adding a resistance on the inverting or non-inverting path based on the value of the left hand side, which we'll call α :

$$\alpha = \frac{1 + \sum_{i=1}^m A_i''}{\sum_{i=1}^n A_i'}, \quad \begin{cases} A_{n+1}' = 1 - \sum_{i=1}^n A_i' + \sum_{i=1}^m A_i'' \\ A_{m+1}'' = -1 + \sum_{i=1}^n A_i' - \sum_{i=1}^m A_i'' \end{cases}$$

2.1 Logic gates

We can move on to digital electronics concept, by introducing **logic gates**. A logic gate is an electronic component which performs some sort of operation on digital signals.

A simple example of a logic gate is an **inverter**. The simplest inverter is given by a *resistive load* inverter.



What happens in the circuit is that if we raise v_{in} , and thus v_{GS} , above the threshold voltage v_{th} , then we get conduction between the drain and source of the MOS transistors, and the circuit acts as an open switch. If we then sample v_{out} , we see the voltage after the fall on R_{pu} (which will be close to zero).

In the other two graphs we see two non idealities of this circuit:

- **Asymmetric** response: the falling edge is much sharper than rising edge, as the R_{DS} of the NMOS is much smaller than R_{pu} ;
- We have **static** power consumption in the on state (when output is at logic 0), as current flows from drain to source:

$$P = \frac{v_{DD}^2}{R_{DS} + R_{pu}}$$

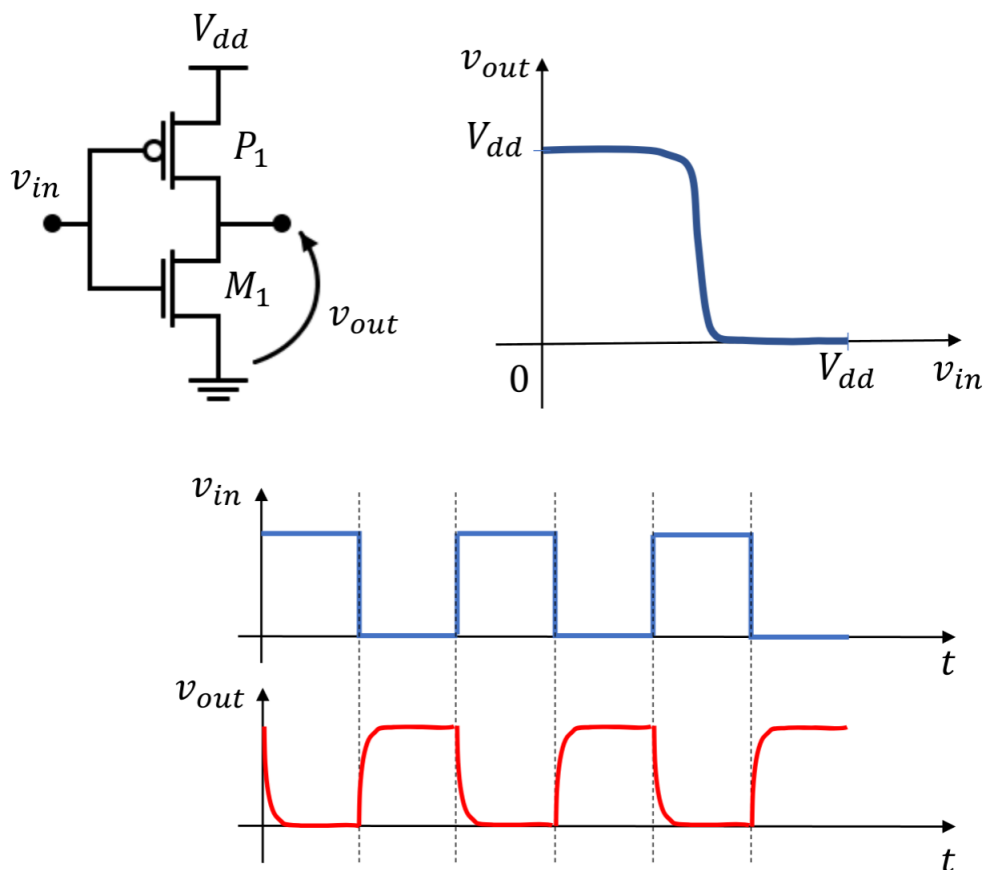
- We also have a certain amount of **dynamic** power consumption, given by the transients happening at state changes. These rely on equivalent capacitances of wires and MOS transistors, and will be studied in more detail later.

We can then see that we need a better design, at least in term of response alignment, and absolutely without static power consumption.

3 Lesson 25-09-26

In the last lesson we introduced the resistive load inverter. We said that the main problem of such a design is the asymmetric response, and static power consumption (as in the output off state we have a closed path from power source to ground, draining power on a resistor). Today we try to solve those problems.

CMOS inverter: a better inverter can be designed by using **Complementary MOS (CMOS)** designs.



We see that in such a design:

- The response is **symmetric** (or mostly symmetric, we can act on the gate sizes to account for mobility effects);
- There is no **static** power consumption (in the ideal case, some leakage does happen);
- **Dynamic** power is present and measurable based on equivalent capacitance of the wire and the MOS transistors:

$$P \approx C_{e1} f V^2$$

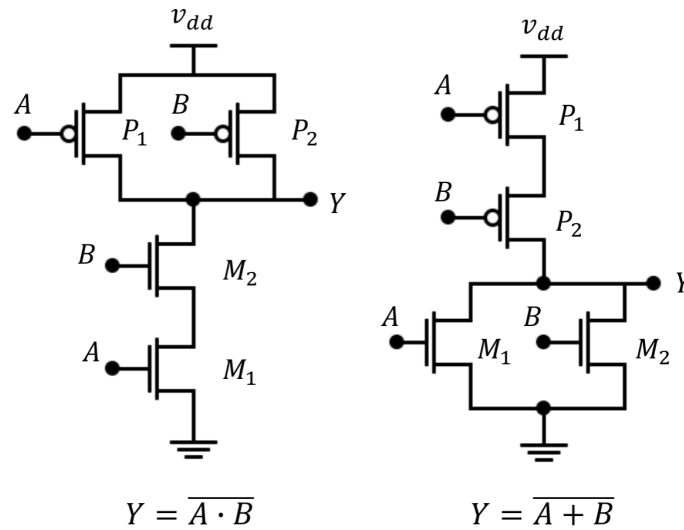
This design is very common in digital circuits nowadays, as it performs much better than the resistive load inverter.

CMOS logic: other logic functions are needed to implement a logic family. For CMOS technology, we can use combinations of transistors in series or in parallel to implement any kind of logic.

In particular:

- In the pull-down branch:
 - Series NMOS implements a NAND
 - Parallel NMOS implements a NOR
- In the pull-up branch:
 - Series PMOS implements a NOR
 - Parallel PMOS implements a NAND

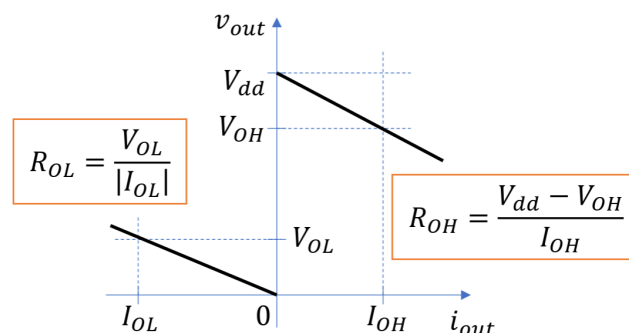
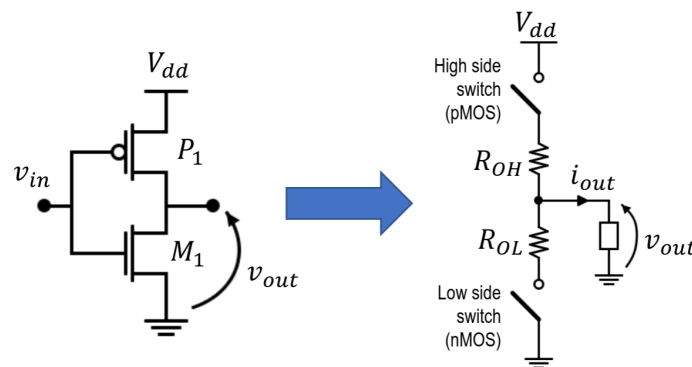
We show a figure of the synthesis of a NAND and NOR circuit in CMOS technology:



Logic gate parameters: we can define input and output parameters for logic gates, regarding both *current* and *voltage*.

With regards to **current**, we can model a CMOS inverter converting the MOS transistors to voltage controlled switches (ideal) with series resistances modeling leakage. This way we see we can define the parameters:

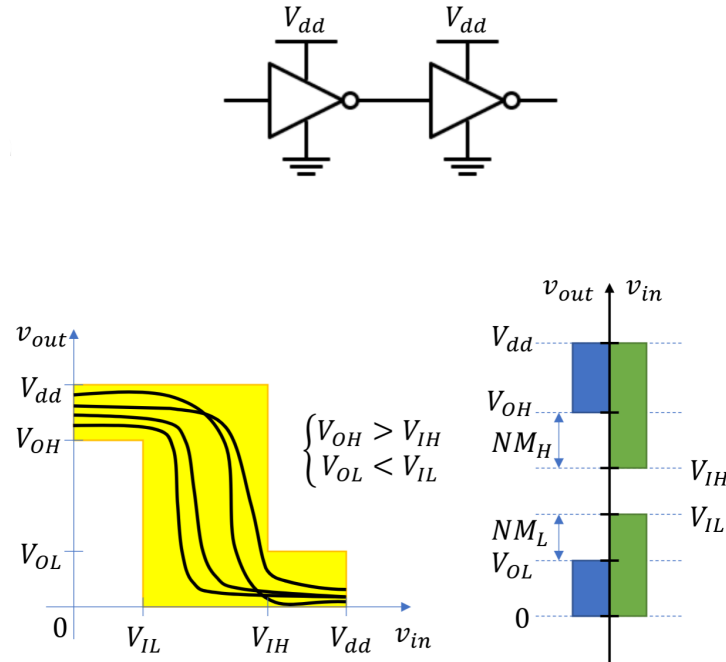
- V_{OH} , which is the lowest output voltage to get a logic high output;
- V_{OL} , which is the highest output voltage to get a logic low output;
- I_{OH} , which is the maximum output current such that the output voltage at logic high is greater than V_{OH} ;
- I_{OL} , which is the maximum output current such that the output voltage at logic low is less than V_{OL} .



From these parameters (given by constructors) we can compute the value of the modeling resistances:

$$R_{OL} = \frac{V_{OL}}{|I_{OL}|}, \quad R_{OH} = \frac{V_{dd} - V_{OH}}{I_{OH}}$$

Now, with regards to input and output **voltages** in stacked gate setups, we can consider the following:



Here we have a pair of connected inverters. We are interested on whether output voltages of the first inverter will be correctly interpreted at the input of the second inverter, so we define the following parameters:

- V_{IH} , which is the lowest input voltage recognized as logic high;
- V_{IL} , which is the highest input voltage recognized as logic low;

From this we can derive stability *conditions* for the whole logic family:

$$V_{OH} > V_{IH}, \quad V_{OL} < V_{IL}$$

where we define the following margins:

$$V_{OH} - V_{IH} = NM_H \quad (\text{noise margin high})$$

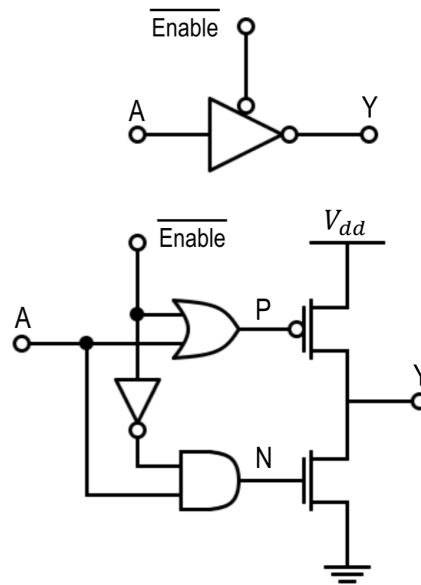
$$V_{IL} - V_{OL} = NM_L \quad (\text{noise margin low})$$

Tri-state outputs are needed to avoid logic gates shorting when connected to the same outputs.

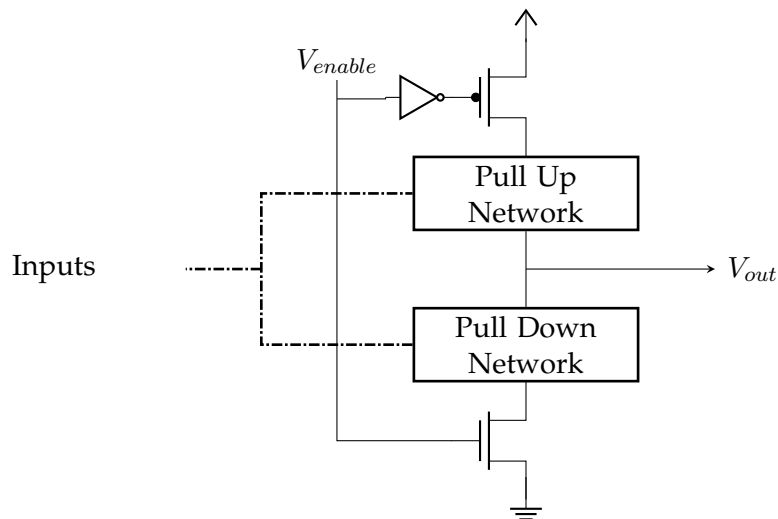
In these designs we have an additional Enable input, which can be *active high* or *active low*:

- When Enable is active, the output is driven normally;
- When Enable is NOT active, both the pull-up (high side) and pull-down (low side) transistors are switched OFF, and the output line is left floating. At this point the voltage level (and consequently the logic level) is set by some other circuit, or remains undetermined.

Below, we show the symbol of a tri-state inverter and a possible CMOS implementation.

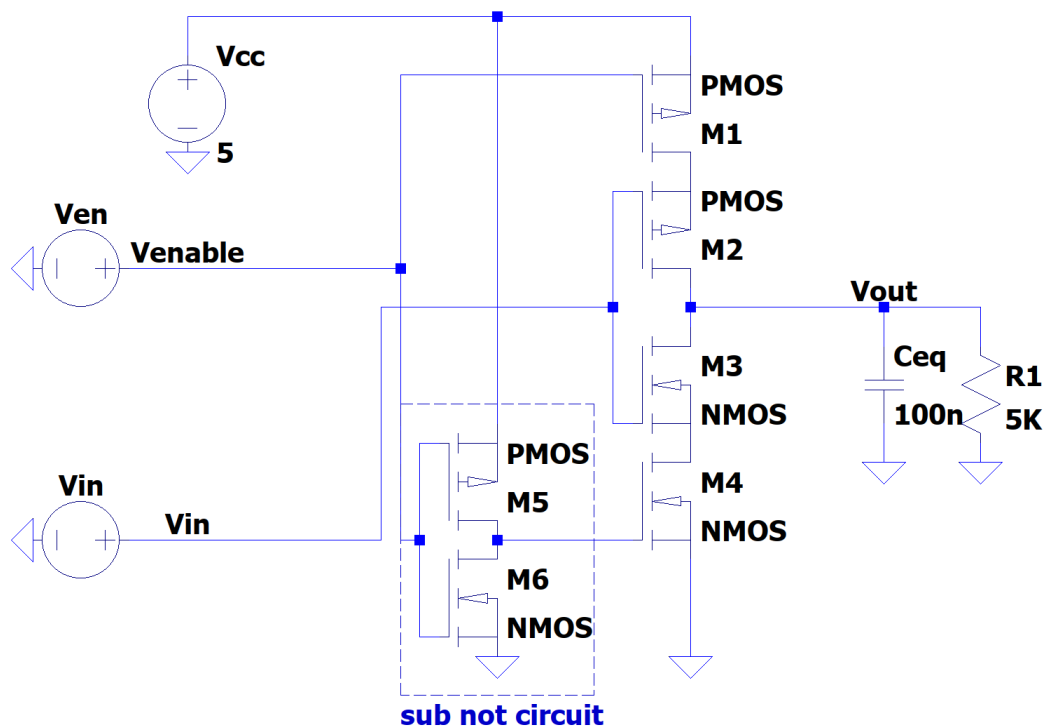


A better design (or which at least uses less components in CMOS logic) for enabled inverters can be obtained using a general method for tri-stating CMOS network outputs. Because the outputs of CMOS logic networks are only given by shortcircuiting to V_{cc} (via the **PUN**, *Pull Up Network*) or to ground (via the **PDN**, *Pull Down Network*), we can fully isolate the output by separating the PUN and PDN's connection to V_{cc} and ground respectively. This is better explained by the following circuit:



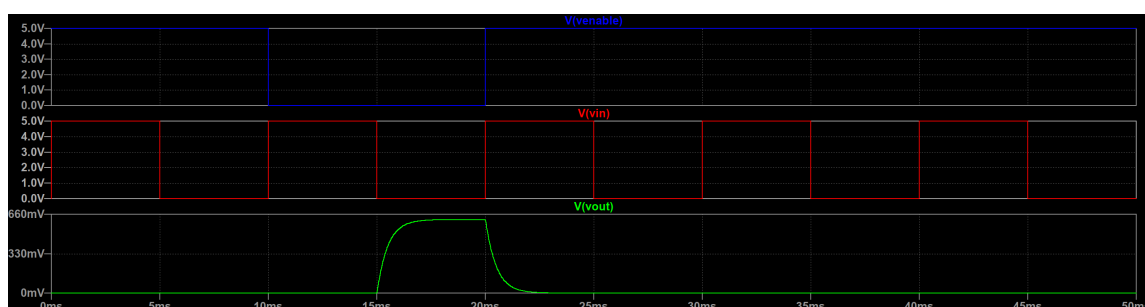
We note that this implementation is active *high*, as when V_{enable} is high the connection to ground of the PDN is broken, and after the inverter (which pulls the enable signal to low) the connection to V_{cc} of the PUN is broken. We can easily turn this circuit into an active *low* implementation by just swapping the position of the NOT gate.

Lastly, we show how this technique can be used to design a CMOS inverter with enable signal, in a simulator such as LTSPICE:



In the circuit, we implement the active *low* version, and implement the NOT circuit manually via 2 other MOS transistors. We also note the RC network at the end of the circuit: this is useful in testing, as it simulates an actual driven line which might have capacitance and resistance (exaggerated here for the example).

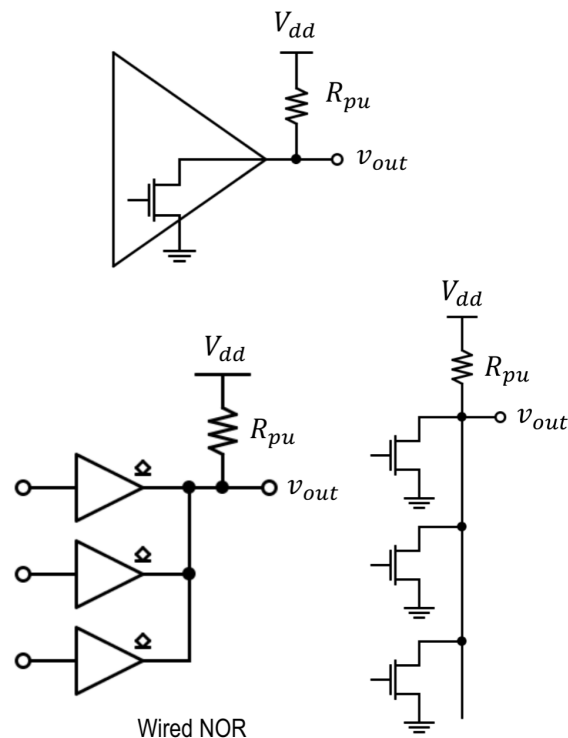
We see how such a network behaves by driving it. In the figure below, after a delay of 10 ms the inverter is enabled, we do a low to high transition, and then it is disabled again. We see how we can temporarily drive the output line, which then goes back to 0 V (by draining through the RC network) after disabling the inverter.



As we said, tri-state outputs are useful when multiple logic gates drive the same line, for instance in a shared bus. All drivers must be in high impedance except one: it is vital to guarantee the above condition to avoid short circuits.

Open collector is another technology which allows to drive the same data line, even without knowing the voltage supply value.

In this setup, there is no pull-up transistor at the output: we can only set the output to logic 0. To set a logic 1, we switch off all pull-down transistors, and use an external pull-up resistor.



This is useful to drive the same line with multiple logic gates:

- No possible short circuit: when all logic gates set a logic 1 the output is at logic 1;
- When at least one logic gate sets a logic 0, the output goes at logic 0. This makes open collector a NOR logic function.

This is slightly easier to implement than the earlier tri-state solution, as we don't have to deal with high impedance or short circuit issues.

Of course, we can imagine an *open drain* setup which works the same, only leaving the pull-down transistor outside of the design and pulling *up* through an external resistor.

3.1 Combinatorial circuits

Combinatorial circuits are the simplest we can see with digital logic circuits. In a combinatorial logic circuit, the logic output depends on the current value of the inputs only (no state).

A combinatorial circuit can be described through a **truth table**, which is a list all possible input combinations, and corresponding output values.

Many ways exist to synthesize a combinatorial circuit:

- The main idea is generally to generate the circuit as a **SoP** (*Sum of Products*), minimizing the number of products, and minimizing the number of literals in each product (which corresponds to minimizing components).
- A possible tool is given by *Karnaugh maps*;
- Another possibility is the *Quine-McCluskey algorithm* for more inputs;
- Often, we just stick to logic synthesis *software tools*, which implement these or other advanced algorithms.

Some other help about this topic can be found in the author's notes at <https://seggiani-luca.github.io/appunti/triennale/2/reti-logiche.html>.

Multiplexers / demultiplexers are circuits which allow to select an input and output just one of them, between different inputs. A one bit multiplexer with 2 inputs has this boolean law:

$$y = a \cdot s + b \cdot \bar{s}$$

where a and b are the inputs, and s is the selection signal.

For multiplexers on multiple bits, we repeat the same structure multiple times; For multiplexers with more than two inputs, we need more than one selection signal (usually built hierarchically with log number of inputs smaller multiplexers). Multiplexers with more than two inputs on multiple bits are built as a combination of the above two cases.

We can summarize this via the following distinction:

- We can expand multiplexers **vertically**, that is add more multiplexers vertically with the same control variables, to extend the width (in bits) of the inputs;
- We can expand multiplexers **horizontally**, that is build hierarchical structures with more control signals, to extend the number of inputs available.

One technique commonly used to build multiplers with MOS transistors is *pass-transistor* logic. Pass-transistor logic involves using an NMOS and a PMOS transistor in parallel to build a (basically) ideal voltage controlled switch. This can implement the logic function of a 2 to 1 multiplexer (above) with less transistors than the equivalent CMOS solution. At this point, more 2 to 1 multiplexers can be expanded as we saw above to have wider or more inputs.

We also note that the inverse of a multiplexer is called a **demultiplexer**, that is a component with multiple outputs, to which we can route a single input based on control signals.

Decoders / encoders: **decoders** are components which decode the binary representation of a number, on n bits, to the same number in one-hot representation, on 2^n bits. **Encoders**, on the other hand, do the reverse job of converting a one-hot representation on 2^n bits in a single n bit encoded value.

More complex components like *priority encoders*, ... can be built, but usually an encoder is in an inconsistent state when multiple input lines are high.

Decoders and encoders are still utility logic circuits, but as opposed to multiplexers and demultiplexers they are more easily implemented with standard CMOS logic, or by cascading CMOS logic devices (such as NOT, NAND and NOR gates).

Digital adders / subtractors are built in combinatorial logic too.

We start from *half adders*: these operate on 1-bit operands, generating a carrying output bit. For *multibit* adders we combine multiple half adders.

Different area and performance can be achieved based on carry computation. For instance, we remember:

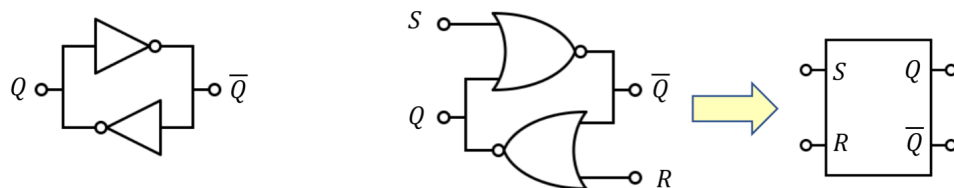
- **Ripple Carry:** this is the traditional method of stacking full adders, where we connect a C_{in} (carry in) and C_{out} (carry out) pin for each adder pair;
- **Carry Look Ahead:** with carry look-ahead we compute the carries in place via a CLA network, and perform the actual sums in parallel. This adds logic but saves time;
- **Carry Select:** with carry select we compute both pairs (without carry and with carry) for each adder, and only at the end of the calculation we choose the correct carry.

Subtractors are just implemented using an adder (using 2-complement notation).

3.2 Sequential circuits

Sequential circuits need a *memory element* to exist somewhere in a circuit. The most popular memory element is given by a pair of active elements in a loop.

- A pair of inverters in a loop provides a memory element, that is an element that is *bistable*: either of the inverters can start out with an high outputs, and the state is converted. The problem of this design is that we can't control it:
- A pair of NANDS in a loop can be used in the same fashion. The input bits that are left out can be used as SET and RESET parameters for the bistable circuit. We call this a **SR latch**.

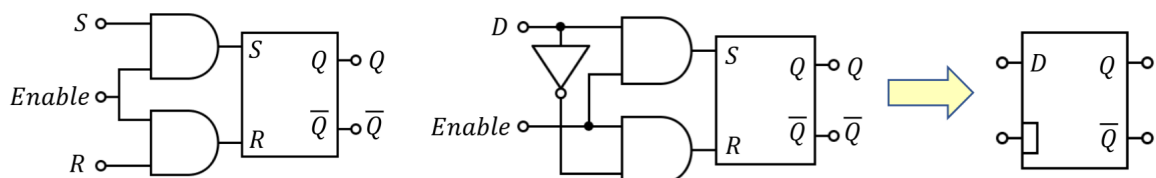


4 Lesson 30-09-26

Last lesson we finished up on sequential circuits, introducing the *bistable circuit* and making it controllable via a pair of NOR ports (implementing an *SR latch*). Let's start again from there.

SR latch w/ enable: let's add enable circuitry to the SR latch. Note that *enable* here doesn't mean setting the output to high impedance (as in tristates), but making the network unresponsive to changes in the inputs.

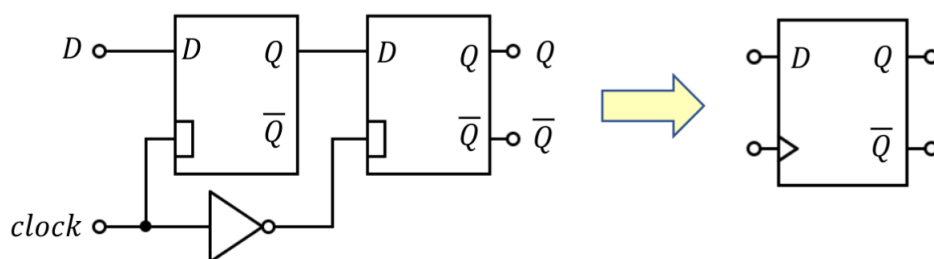
The circuits we could get are the following:



D latch: let's add enable circuitry to the SR latch.

Note that we decided to simplify the SR latch circuitry in the middle circuit, by setting S to a data value D , and R to its negation. This allows us to build what's essentially a **D latch** (last circuit in the row).

D flip flop: this is one last memory circuit we can build, which is even more convenient than the past two.



In the figure we note the master/slave implementation, but there are many others that can be built. The idea of the D flip flop, no matter the implementation, is that the output value Q follows the input D *only* on the rising edges of the clock signal. This makes D flip flops non *transparent*: a change in the input isn't transmitted directly to the output (but only transiently during clock rising edges): in other words, if the circuit oscillates, it oscillates synchronized to the clock. This makes this circuit extremely useful in sequential networks as a memory element (for instance in *registers*).

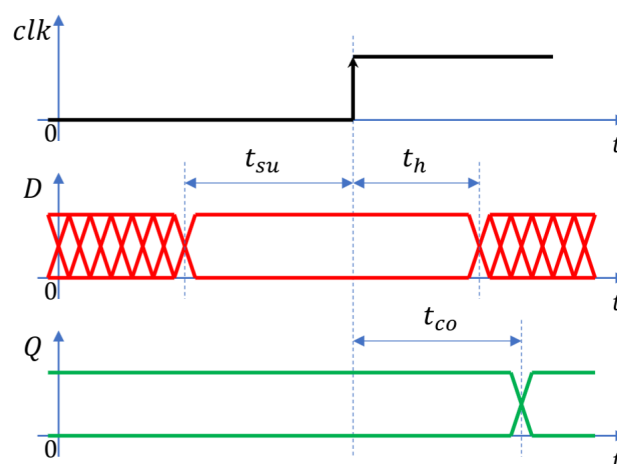
Other characteristics: let's note other characteristics of sequential networks.

Firstly, when talking about flip-flops and latches, we note that there are usually other inputs provided:

- **Asynchronous clear** (or **reset**): sets the output to 0, without waiting the active clock edge;
- **Asynchronous set**: sets the output to 1, without waiting the active clock edge;
- **Clock enable**: masks the clock input.

These signals are often active low.

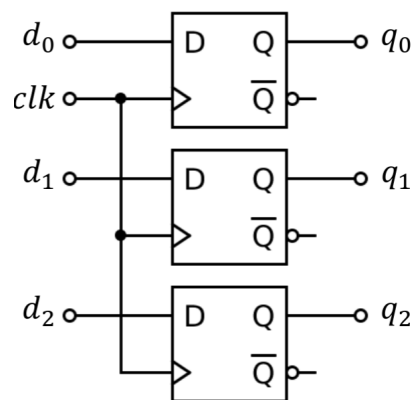
Other important characteristics are the **timing specifications** of flip-flops:



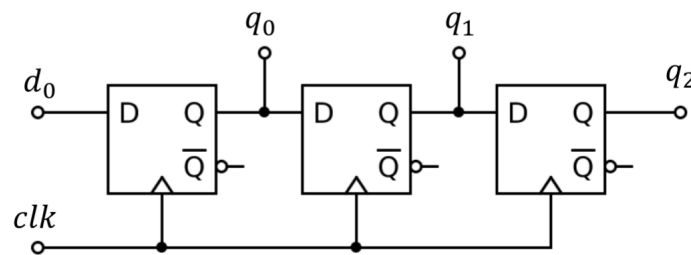
- **Setup** time: the time the data input has to be stable *before* the rising edge, for the flip-flop to correctly recognize its value;
- **Hold** time: the time the data input has to be stable *after* the rising edge, for the flip-flop to correctly recognize its value;
- **Clock to output** time: the time the flip-flop takes to propagate its input to its output after a clock rising edge.

These values are often in the order of microseconds, and directly determine the maximum clock speeds we can achieve.

Registers: we introduced them, now let's look at them. A **register** in the electronics sense is nothing more than a combination of multiple flip-flops with a common clock and different data inputs. This allows registers to store whole *words* of data, of arbitrary size. Let's look at a 3 bit register as an example.

**3-bit Register**

Shift register: a *shift register*, often called a *delay line*, is like a register which takes in a single bit of data and propagates it through a set of flip-flops.

**3-bit SIPO Shift Register**

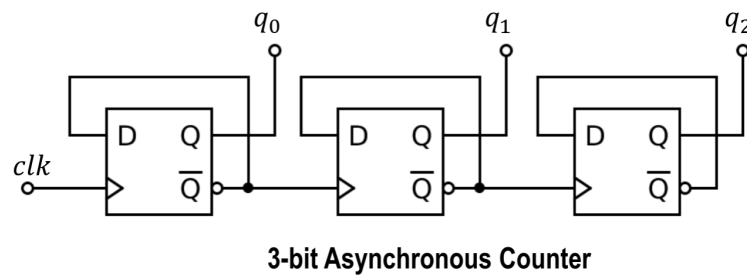
There are different kinds:

- Serial In, Parallel Out (**SIPO**);
- Parallel In, Serial Out (**PISO**): these need extra control inputs such as:
 - Parallel Load (PL)
 - Load Enable (LE)
- Serial In, Serial Out (**SISO**): these often come with parallel out, as well.

All shift registers may have a shift enable signal.

Counter: there are 2 kinds of counters:

- **Asynchronous counter:** the clock propagates from one FF to the next. This means that the most significant bits are updated later than least significant bits. This is undesirable as it entails *data races* along the FFs. Still, it is the one we show an example of as it's the simplest;
- **Synchronous counter:** the same clock is provided to all FFs. Internal logic used to compute the input to each FF: this means all output bits are updated at the same time.



This being said, there are other characteristics which determine the behavior of a counter:

- Up or Up/Down counters;
- Binary or Decade counters;
- Parallel Load to initialize counter value;
- Carry input and output to cascade counters;
- Asynchronous reset.

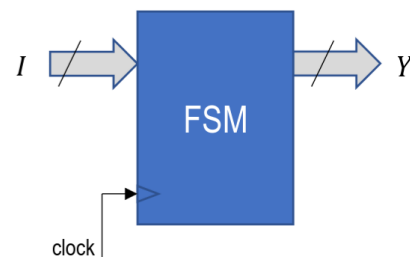
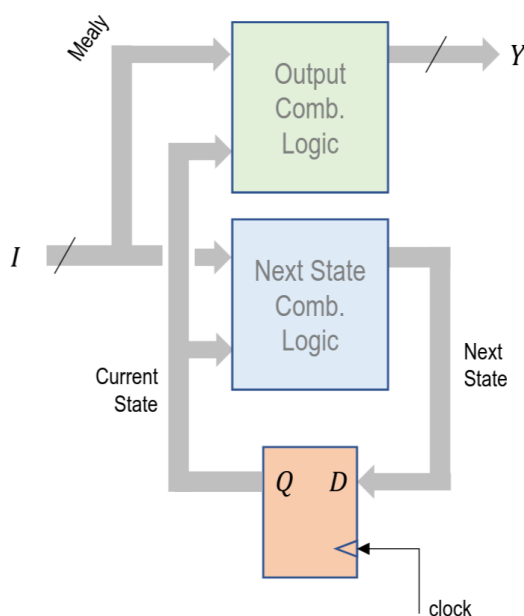
FSM: an **FSM** (*Finite State Machine*) is a graph structure consisting of *nodes*, corresponding to **states**, and *edges*, corresponding to **state transitions**. This way, a state machine defines all possible states of a digital systems and all the ways states can transition one to another.

A FSM description can also entail a network's outputs:

- Edges may have output values (*Mealy model*);
- States may have output values (*Moore model*);
- Transitions are synchronized with an external clock signals (*retarded* or *synchronized* Mealy and Moore models).

State machines can formally be described via two truth tables:

- One for *state transitions* (next state), with respect to inputs and current state
- One for *outputs*, with respect to current state (Moore) and inputs (Mealy).



To implement a state machine as a circuit we first need the *specification* of that state machine, which we said is either given by truth tables or state transition graphs. Then, we can encode the internal state in a special register (the **STAR**, or *STatus Register*). The STAR register has to be as big as the base-2 logarithm of the state space (in *binary* encoding, as big as the state space for *one-hot* encoding).

At this point the two truth tables become two combinational logic circuits:

- One that takes in the current state (Moore) and/or the current inputs (Mealy) and returns the output of the network;
- One that takes the current state and the current inputs to determine the next state (which gets fed back to the STAR register).

4.1 VHDL

VHDL (*VHSIC Hardware Description Language*) is a hardware description language that can model the behavior and structure of digital systems at multiple levels of abstraction, ranging from the system level down to that of logic gates.

- It is used to *describe* the structure and the behavior of a **digital system**;
- The description can be used to *simulate* and *synthesize* a **digital circuit**.

The nice part of VHDL is that the syntax is similar to a programming language.

The basic design element of a VHDL description is an **entity**, which is a design unit comprised of:

- The **entity** specification, which acts as a black-box interface for the component:
 - **Name** of entity;
 - **Inputs**;
 - **Outputs**;
 - **In/out** ports;
- The **architecture** specification, which describes the entity's behavior and/or internal structure. VHDL allows for a number of specification styles:
 - **RTL** (*Register Transfer Level*): uses *concurrent signal assignments*.
These are *boolean expressions* assigned to a signal, executed in *parallel*, so without any implied order. Not that lists of concurrent assignments are *not* sequential in execution. High level operators such as arithmetic, logic and conditionals can be used;
 - **Structural**: similar to a schematic diagram, but using a textual programming language.
We first declare *components*, then *signals* to interconnect those components, and lastly *map* signal and input/output ports to components. This allows for a hierarchical description of a circuit (Top/Down or Bottom/Up);
 - **Behavioral**: uses a process, which is a sort of an endless loop, specified in a programming language-like way.
In most cases it uses an implicit *wait statement* on a *sensitivity list*:
 - * The process suspends when it reaches the end
 - * Resumes when any signal on the sensitivity list changes

Instructions in the process are, this time, in fact *sequential*, but may still be implemented in parallel to increase performance if the compiler finds it is feasible. Still, we note that signal assignments happen only *at the end* of a certain process. Higher level constructs such as if or case conditional expressions are available, variables can be defined and assignments to variables are immediate.

4.2 Introduction to embedded systems

This course and the entire degree program are focused on **embedded systems**, so it might be natural to ask *what* an embedded system even is. An embedded system is an electronic system, that is a system made up of:

- Hardware components;
- Software components;
- Communication systems.

designed to be part of a larger system, not necessarily electronic:

- Vehicles (cars, planes, ...);
- Industrial systems;
- Home appliances.

This means that an embedded system:

- Is, in fact, **embedded**, i.e. it lives in a larger environment;
- It should **react**, typically in real-time, to external stimuli;
- It is dedicated to a particular **application** (single task).

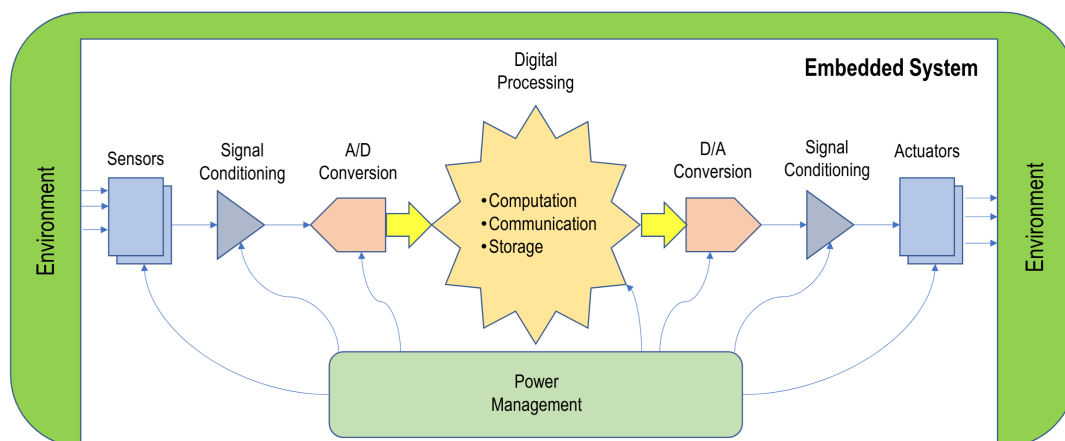
This gives designers the opportunity to highly optimize and adapt to different situations, in highly constrained environments.

Designing embedded systems, in this regard, is difficult because:

- The design space is extremely large;
- It requires skills from many very different domains;
- It requires to simultaneously optimize numerous design metrics.

4.2.1 Architecture of an embedded system

Let us look at the typical components of an embedded system:



- We already know that an embedded system lives inside an **environment**, from which it *senses* data and which it modifies by *acting* on it;
- Data will be obtained from **sensors** that return electrical values proportional to physical quantities in the environment;
- Data will be **conditioned**, which means *amplified, filtered*, or anyways manipulated in order to be more useful to the embedded system;
- Data will be converted from an analog signal to a digital value ready for computation by an **A/D converter**;
- The central part of the embedded system performs **digital processing** of the recovered data from the sensors, signal conditioning and A/D conversion parts. What comes out of this phase is digital signals which will be used to control the *actuators*;
- Data will be converted from the digital processing stage back into the analog domain by a **D/A converter**;
- Again, we will have some amount of **conditioning** on the output signal;
- The conditioned signal will drive **actuators**, which can actually affect the environment;
- All of these components need **power management** to be correctly powered by some power supply (often batteries, but also wall power, ...).

4.2.2 Design metrics for embedded systems

A **design metric** is a measurable feature of a system implementation. Common metrics are:

- **Unit cost** U : the monetary cost of manufacturing each copy of the system;
- **NRE cost** N : one-time monetary cost of developing (designing) the system;
- **Total cost** T : total cost of producing N_{unit} tests. Calculated as $T = N + U \times N_{unit}$;
- **Per-product cost** p : cost of a singular product, with amortized NRE costs. Calculated as $p = \frac{T}{N_{unit}} = \frac{N}{N_{unit}} + U$. Keeping the per-product cost low entails spending as little as possible on NRE (as these costs get amortized over units produced) or producing as many units as possible. Of course the unit cost U plays a significant role: lowering is sometimes useful at the cost of increasing N ;
- **Time-to-market**: it is the time required to develop a product to the point it can be sold to customers. Smaller time-to-market means less losses due to delayed market entry (for instance, when competing with other products);
- **Performance**: it can be measured in many ways. For instance:
 - **Latency**, or *response time*, the time between task start and task end;
 - **Throughput**, tasks per second, usually $\frac{1}{\text{Latency}}$.

Parallelism is a way to increase performance. Usually, N parallel blocks allow to increase throughput to $N \times \text{Throughput}_{\text{Serial}}$ (unless Amdahl's law comes into play);

- **Technologies**, that is technical processes, methods and knowledge, are in some way a design metric.

4.2.3 Technologies

Let's look at a few **technologies** which we have available for the design of embedded systems. We noted the design space is very large, so it is useful to know about them.

Processor technology

Let's look at **processor** technologies.

- The *General Purpose (GP)* processor, as found in desktop PCs, is the classic programmable micro-processor we are used to.
 - They are not optimized for any specific application;
 - They are often cheaper than general purpose technology: billions of units are sold every year;
 - Flexible, low time-to-market: software is easy to update and modify;
 - No high performance compared to a dedicated design;
 - Not power efficient
- *Embedded* micro-processors, though, are different from PC micro-processors. For instance, they are usually within a *System-On-Chip (SoC)*, which includes needed components and a number of peripherals.
- A method of implementing embedded processors is *Application Specific Instruction Processor (ASIP)*. This entails:
 - A general purpose processor with specific hardware for a certain application domain, for instance **DSP** (*Digital Signal Processing*);
 - Dedicated to signal processing application;
 - Possibly dedicated hardware to drive and read external pins;
 - Cheap;
 - Flexible, low time-to-market;
 - Higher performance and better power than a GP processor.
- Another method is *Single Purpose (custom) Processor*. This entails:
 - Digital hardware designed specifically for a certain application;
 - Best performance;
 - Best power consumption;
 - Small size;
 - Long time-to-market;
 - Very high NRE costs;
 - Low flexibility;
 - Good for large volume productions and for extremely high performance designs.

IC technology

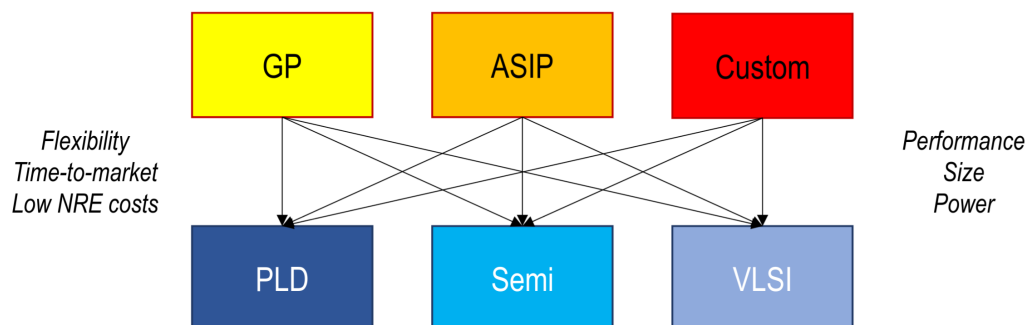
ICs (*Integrated Circuits*) also have different technologies with different use cases.

- *Full Custom (VLSI, Very Large Scale Integration)*:
 - The integrated circuit is entirely designed and optimized for a particular application;
 - Transistor sizing, placing and routing;
 - Best for performance, size and power consumption;
 - High NRE costs;
 - All masks in the production process are dedicated to a single design;
 - Long time-to-market;

- Currently used to design micro-processors (GPs and ASIPs), memories and off-the-shelf Programmable Logic.
- *Semi-Custom* (Gate Array / Standard cells) technology:
 - The lower layers are fully or partially built: design entails placing of large blocks, routing;
 - Good performance, size and power consumption;
 - Lower NRE compared to a full custom implementation;
 - Some masks are shared among more than one design;
 - Still, high NRE costs (tough lower than VLSI);
 - Long time-to-market.
- *Programmable Logic Device (PLD)*:
 - The integrated circuit is fully built by the manufacturer: the embedded system designer buys the IC. The PLD can be customized after fabrication for a specific application;
 - Logic functions and interconnections are programmable;
 - Low NRE costs;
 - Fairly good performance and power consumption, but worse than full custom and semi-custom;
 - High unit cost;
 - Large size.

Orthogonality

Processor and IC technologies are **orthogonal**: the design space is very large, and takes very long to explore. We show a figure showing the different processor and IC technologies we considered, and their characteristic (low N vs high U , or low U vs high N).



Early design decisions are difficult, and it is useful to make use of modern design technologies to evaluate different solutions.

It can also be of use to note that the design phase of a product is also a field where more than one methodology (i.e. a technology and its associated techniques) can be used. This is particularly important in companies, which might have long historical experience with one methodology instead of another. Using well-trusted methodologies or using new ones is also an important factor to consider.

4.3 Memories

Memories are the first component of embedded systems we will encounter. They exist in the digital domain, and more specifically in the storage component of the digital processing part of an embedded system.

In general, there are different kinds of memory, which differ in types, operations available, interfaces, timings and composition of different memories to make bigger ones.

We know the usual types of memory:

- **ROM** (*Read Only Memory*), that is EPROMs, ...
- **RAM** (*Random Access Memory*), that is SRAM, DRAM, ...

We will also note the use of memories as *cache memories* and *memory controllers*.

5 Lesson 01-10-26

Last lesson we started talking about *memories*.

5.0.1 Digital memories

The kinds of memory we mainly care about are **digital** memories. These are **ordered** collections of data, where not all data is necessarily available or accessed concurrently.

Data, in our sense, is made out of n -bit *words*:

- A memory contains N_{word} words of identical size;
- Each word as a width of n_{bits} ;
- Each bit is a binary storage element (such as a flip-flop, even though flip-flops are usually not used in digital memories).

Words are contiguous in memory, and each word is identified by a unique index we call **address**. Addresses usually start from 0 up to $A_{\text{last}} = N_{\text{words}} - 1$.

Sizing

Since we usually address with binary counters, the number of words is always a power of 2: addresses can be expressed on:

$$k = \log_2(N_{\text{words}}) \text{ bits}$$

The number of bits in a word, on the other hand, can be any number (though usually it also is a power of 2, allowing memories to store their own addresses). Sometimes it is useful to include extra bits after 2^n bits, for parity checking or other **EDC** (*Error Detection Code*) techniques.

We can call the **size** of the whole memory the product of words per bits:

$$\text{size} = N_{\text{words}} \times n_{\text{bits}}$$

This can be interpreted as the total number of bits in the memory.

Two memories can have the same size but different organization: for instance, the same total number of bits, but different word size and number of words. The organization affects the memory interface (number and kind of access pins).

We shall also note that *external* access also affects the memory interface:

- Parallel, Serial;
- Random, Sequential;
- Special access modes.

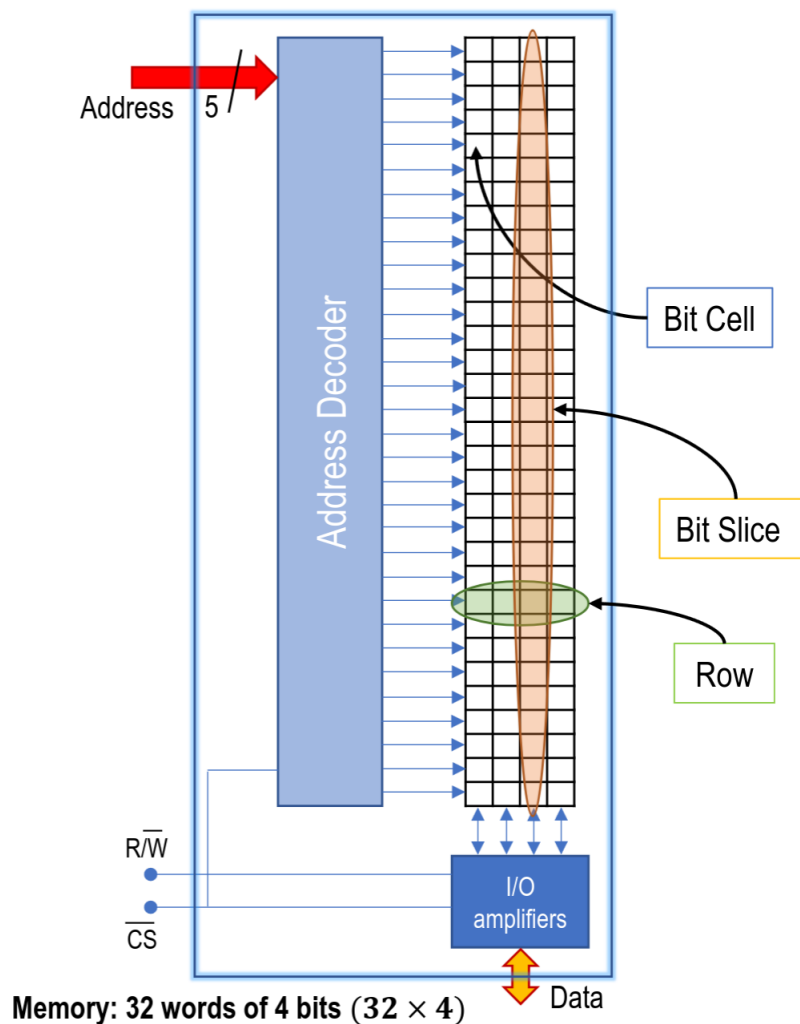
5.0.2 Sense amplifiers

We also quickly explain what *sense amplifiers* are: the signals stored within memories are usually too weak to interface to the external world. For this reason we use amplifiers. These are used to compare a *bit line* (more later) to a *reference line*, and get a biased signal. Note that the reference line can be another bit line!

5.0.3 Logical structure of digital memories

The **logical** structure of a memory, as we explained it, is an **array** of bit cells:

- One row for each word;
- One bit slice for each bit in the word.



Some *address decoder* is used to select a single row of the memory: from k address lines to 2^k word select lines. At the word interface, a sense amplifier and I/O circuit are used to interface to the external world.

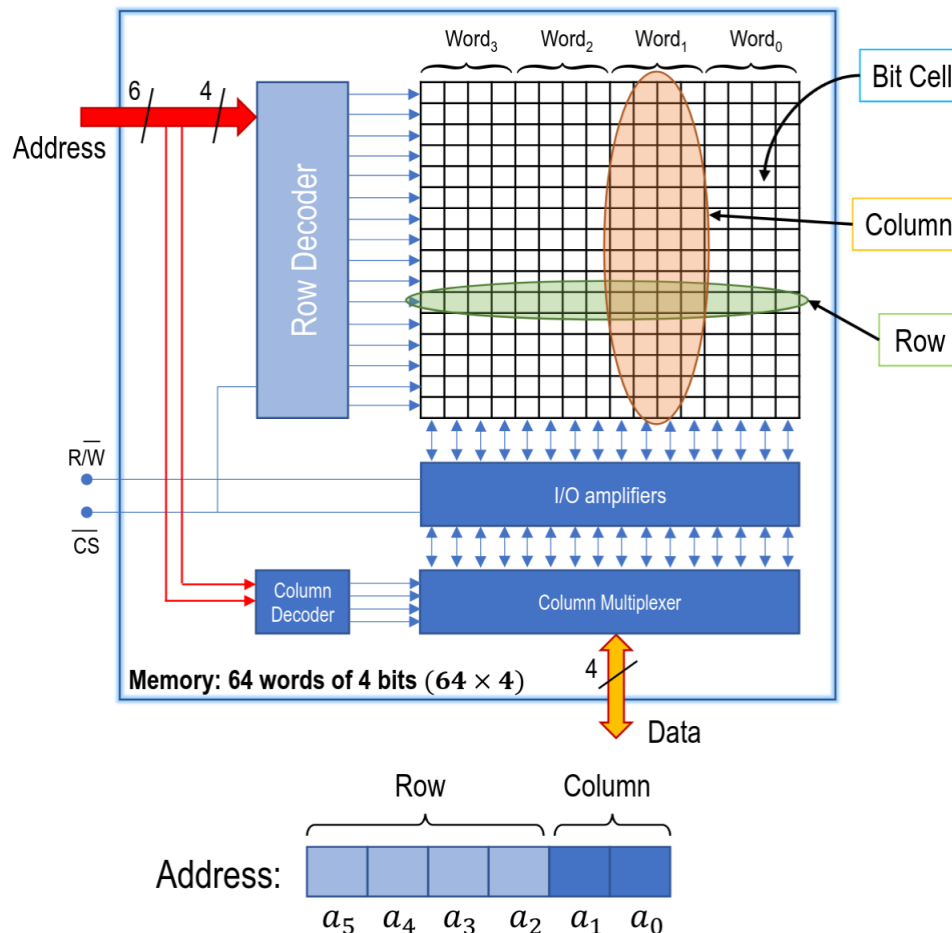
This is not the ideal approach though, as:

- The aspect ratio is unusual and difficult to work with;
- The size of address decoder is exponential on the number of address bits;

- The memory is *slow*: large parasitic capacitance on bit lines increases propagation delays.

5.0.4 Physical structure of digital memories

We then move on to defining the **physical** structure of a real memory. The idea is to not have a *linear*, but a **square** array (or matrix) of word cells.



So we find that the real structure is still an array of bit cells, organized in a square array (optimal aspect ratio). This means that we can have more than one word for each row: one bit slice for each bit in the word, and one column for each word.

Our address decoder has to become a *row decoder*, and we need an additional *column multiplexer* after the I/O amplifiers to select the correct word from a row. As we can see, this is achieved by splitting the address:

- The *upper* side of the address is used to select a row;
- Within a row, the *lower* side of the address is used to select a column.

The sense amplifier and I/O circuit are still used to interface to the external world.

From the user interface, the interface is the same, but the internal organization is different (and way easier to implement physically).

5.1 Memory interfaces

We can now talk about the **interfaces** of memories. Usually these are device pins used to access the memory content, write/read it, ... They are grouped based on their function:

- **Address** bus: identifies the memory location (the index of the word) where an operation is performed. A straightforward address bus is comprised of $k = \log_2(N_{\text{words}})$ lines;
- **Data** bus: provides the data to/from the memory word. Can be bidirectional, or split in input/output monodirectional buses. A straightforward data bus is comprised of n_{bits} lines;
- **Control** bus: this contains the control lines used to perform operations on the data stored in memory. A lot of control lines can be defined, for instance:
 - Clock;
 - Reset;
 - Chip select;
 - Output enable;
 - Write enable;
 - Read/write select;
 - ...

Memory interfaces (as the name suggests) are designed for interfacing a memory to a digital circuit. The digital circuit shall connect to all pins of the memory, drive the inputs according to the specifications in the datasheet, and read the outputs. All of this must be done while complying with the timing diagrams.

- In a *custom* digital circuit, the interface is usually designed specifically for the given memory;
- In a *standard* digital circuit, there is a generic memory interface: it may require external components to correctly drive a certain memory.

5.1.1 Memory operations

This can be a good moment to note that the memory **operations** available are:

- Read: given an address, returns the word contained in memory at that address;
- Write: Given a word and an address, stores the word in memory at that address;
- Match: Given a word, searches the memory for that particular word content and returns the address. This is specific of Content Addressable Memories (**CAM**, *Content Addressable Memory*, used in caches, for instance);
- Erase: initializes the memory to a pre-defined content. Typical of some memories;
- Other operations are sometimes available, such as: Burst operation, Partial operation, Precharge, Refresh, ...

5.1.2 Memory protocol

Data access via the address, data and control lines is performed by an external component on the memory via a predetermined **protocol**. The protocol allows organizing the memory to have better performance and area size:

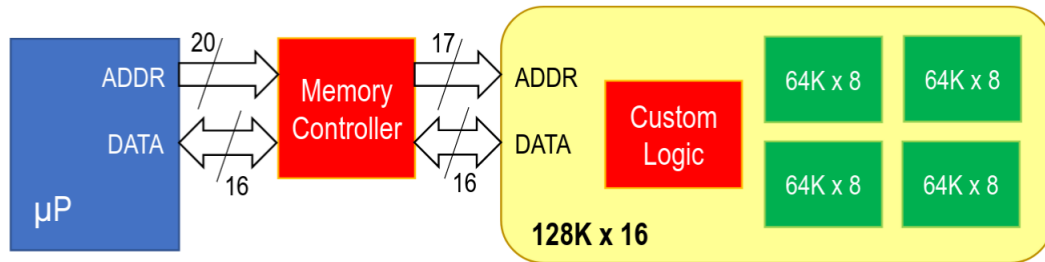
- It defines the sequences of signal activations to correctly perform different operations;

- It defines the timing constraints.

A memory protocol is often described as a timing diagram in the device datasheet.

5.1.3 Memory composition

One of the most important concepts regarding memories at the logic level is memory **composition**. This entails combining multiple memory chips of a given size s_i to obtain a single addressable memory of size $\sum_i s_i$.



We have 3 main cases:

1. Let's assume the case where the required memory needs more words than those available in a single memory chip. For compatibility, we also assume that word width is the same.

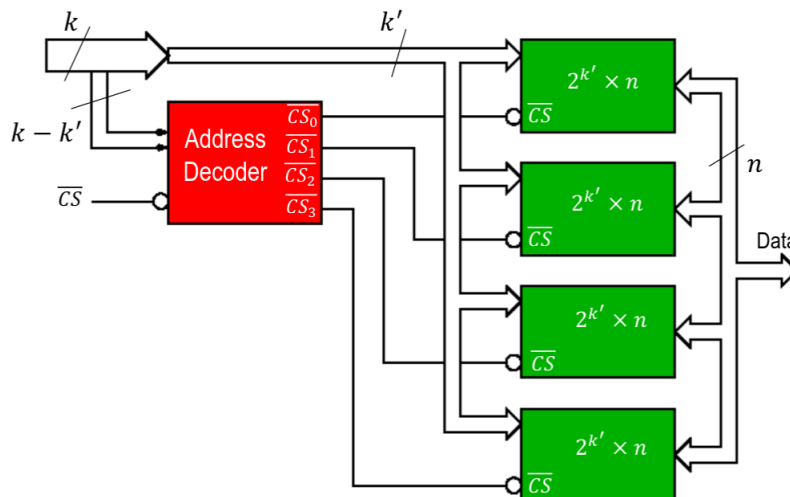
This means that addresses are size k , with each memory chip having addresses of size k' and:

$$k > k'$$

The number of memory chips needed is given by:

$$n_{\text{chips}} = \frac{2^k}{2^{k'}} = 2^{k-k'}$$

We want only one of these chips to be active (to *respond*) for a given address range: this means implementing custom logic to drive the $\overline{CS}$ (*active low Chip Select*) signal of the memory chips. This custom logic is basically a decoder, controlled by the $k - k'$ additional address lines (over k') and enabling $2^{k-k'}$ (which have to alternatively drive one singular data output bus, a.k.a. tristate outputs).



2. Now there's the case where the required memory needs words with more bits than those available on a single memory chip. For compatibility, we also assume that N_{word} is the same.

This means that words are to be size n , with each memory chip having words of size n' and:

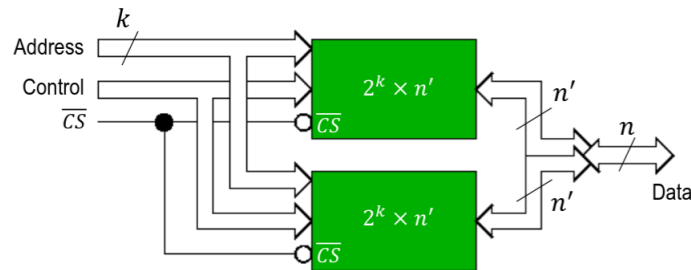
$$n > n'$$

The number of memory chips needed is given by:

$$n_{\text{chips}} = \text{ceil} \left(\frac{n}{n'} \right)$$

At this point the composition is trivial:

- All memory chips share the same address,;
- All memory chips share the same $\overline{CS}$ chip select signal;
- Their data busses are combined (concatenated) to form a larger data bus.



3. The last case is the combination of the first two: required memory needs more words than those available in a single memory chip and needs more bits in a word than those available in a single memory chip.

This means that there are: • k required address lines, but k' address lines in a memory chip, with $k > k'$; • n required data lines, but n' data lines in a memory chip, with $n > n'$.

We need more than one memory chip (of course), and exactly:

$$n_{\text{chips}} = \text{ceil} \left(\frac{n}{n'} \right) \cdot 2^{k-k'}$$

5.1.4 Memory controllers

Lastly, let's spend a note on **memory controllers**. These can be useful, for instance, when general purpose processors are interfaced with off-the-shelf memory chips: the amount of address lines might not match!

Now in this situation, without a memory controller, the microprocessor accesses the same location in memory with multiple addresses. Also, some pins are left floating.

A *memory controller* is used to translate addresses and activate the memory. The memory is mapped to a certain address range in the microprocessor's addressable space: the range does not necessarily start at address 0: the most significant bits can be used by the memory controller to select the appropriate memory. Though, it's good to say that the address range is usually aligned to a power of 2 to simplify the logic.

We know this far too well from our experience with *compiler* tooling: the compiler for the microprocessor should know the chosen address range, so that it generates code that correctly accesses the memory.

Lastly, we note that the memory controller may be integrated inside the microprocessor.